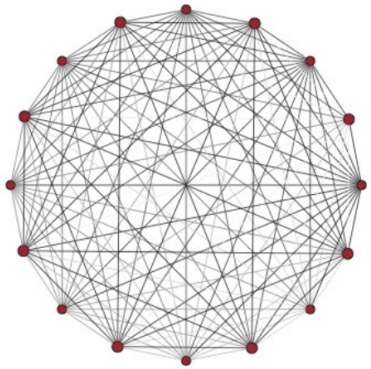
بسم الله الرحمن الرحيم

Éléments de la théorie des graphes

Module : Recherche Opérationnelle

Année 2024-2025

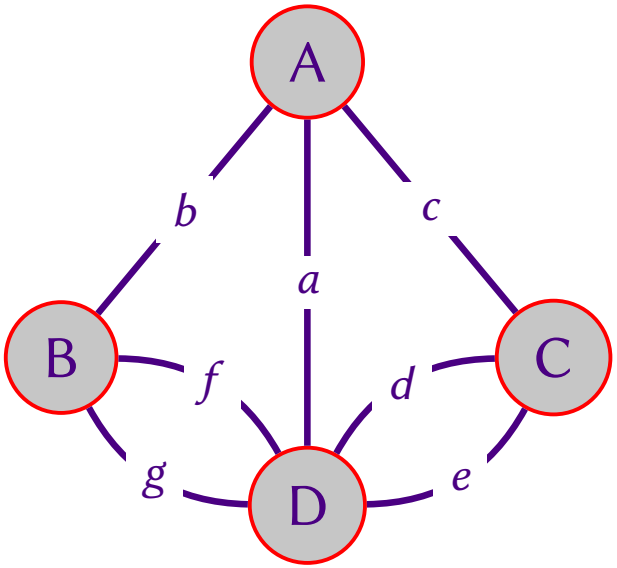


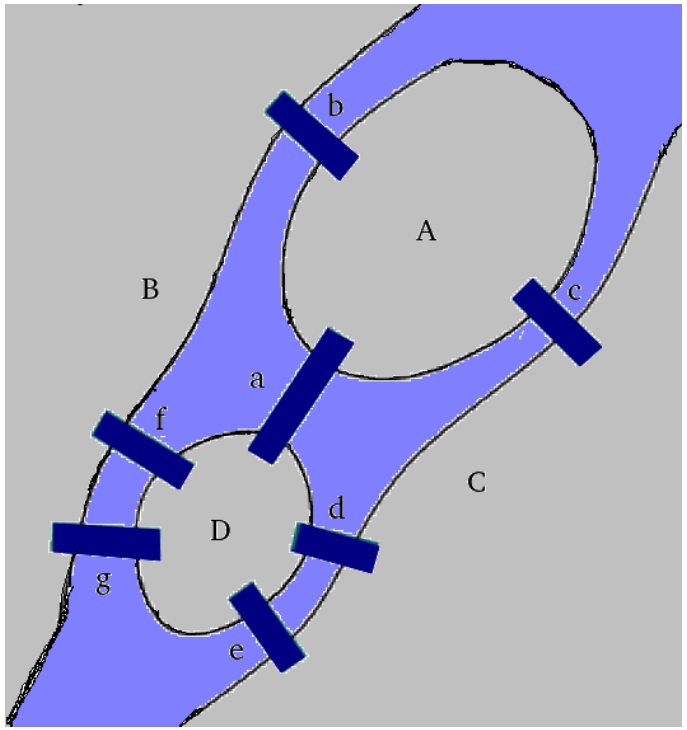
Introduction

Il est souvent naturel d'abstraire un problème ou un objet en traçant sur une feuille des points, représentant des objets élémentaires du problème, et en les reliant par des traits ou des flèches mettant en valeur une relation entre ces points. On dit « Un schéma vaut mieux qu'un long discours ».

Introduction

Un exemple classique de problème sur les graphes est le problème de la ville de Königsberg (plus récemment Kaliningrad), traité par Euler (1736). Cette ville touristique est traversée par la Pregel qui coule de part et d'autre de l'île de Kneiphof.





Elle disposait de sept ponts et un circuit de visite permettant de parcourir ces 7 ponts, sans jamais passer plusieurs fois par le même pont, était très désirable.

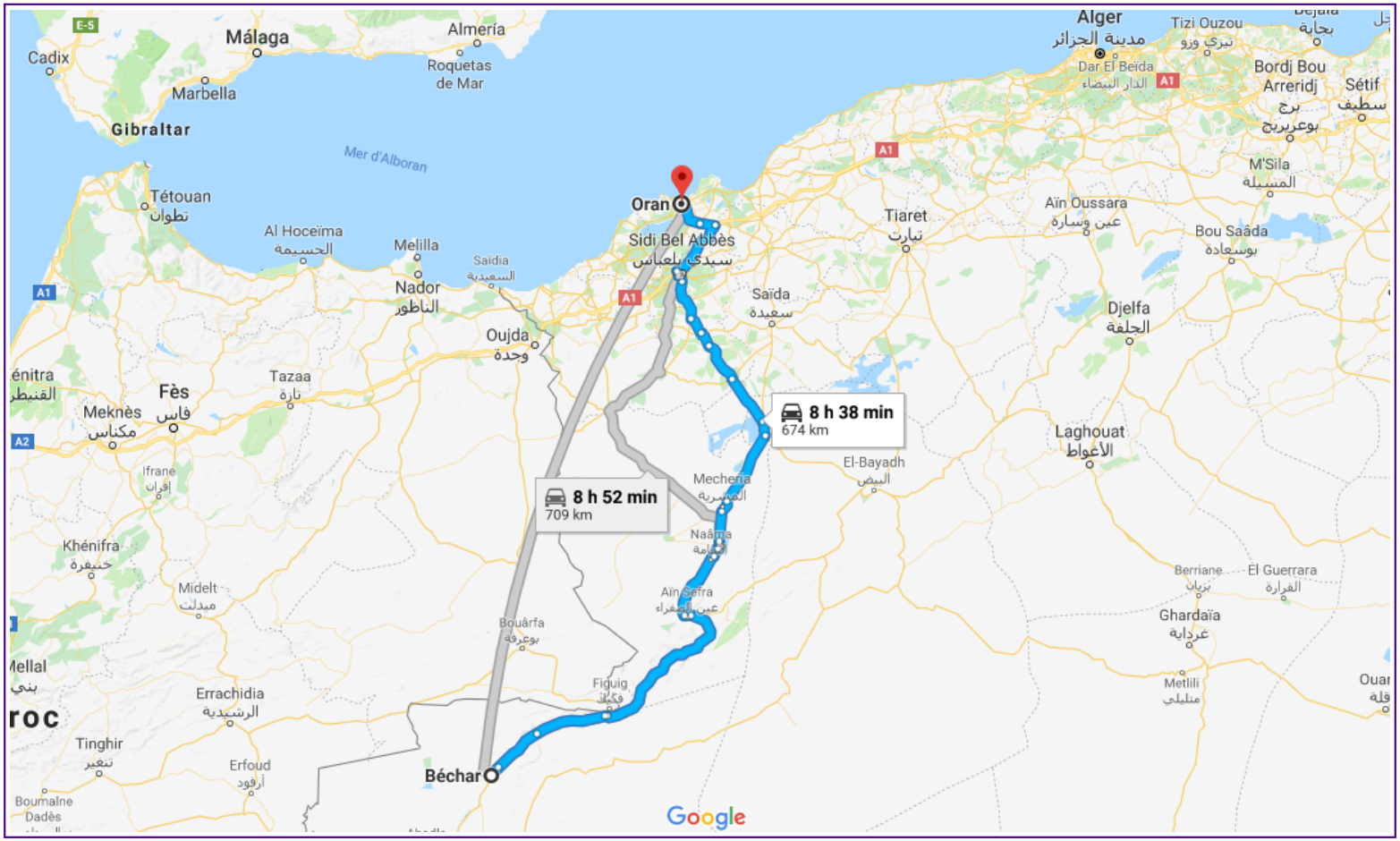
Introduction

Ce problème peut simplement se modéliser sous la forme de graphe. On associe à chaque région géographique un point (ou sommet) et chaque pont est représenté par un trait (une arête) qui relie deux régions, donc deux sommets.

Le problème est alors d'identifier un chemin dans le graphe qui passe une fois et une seule par chaque arête. Il s'agit de la recherche d'un circuit eulérien et dont on sait qu'il n'existe que si chaque sommet est connecté à un nombre pair d'autres sommets. Ce n'est pas le cas ici et donc on sait qu'il n'existe pas de solution au problème de la ville de Königsberg.

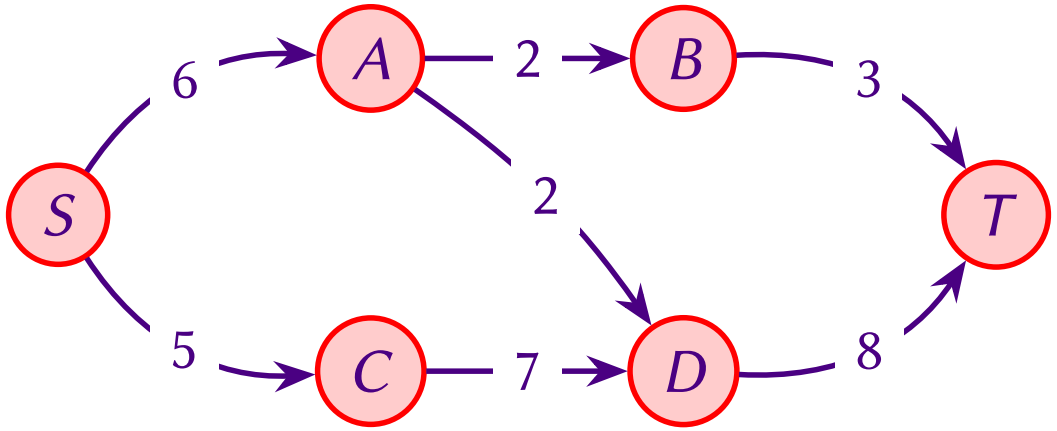
Le plus court Chemin :

consiste à trouver pour une ville de départ donnée et une ville d'arrivée donnée le chemin le plus court qui relie ces deux villes.



Flot maximum :

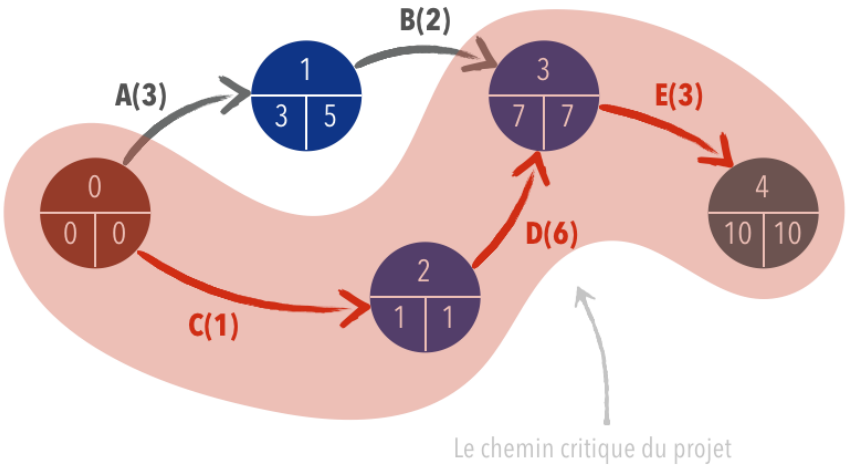
Soit des châteaux d'eau ayant un débit constant. Ils desservent un certain nombre de villes, chacune ayant des besoins quantifiés constants. L'eau est acheminée à travers des conduits dont le débit maximum est connu. Le problème est de trouver un moyen de satisfaire au mieux les demandes de chaque ville. En d'autres termes, essayer d'apporter le plus d'eau possible vers les villes.



Exemples de Pb modélisés par la théorie des graphes

Ordonnancement / planification :

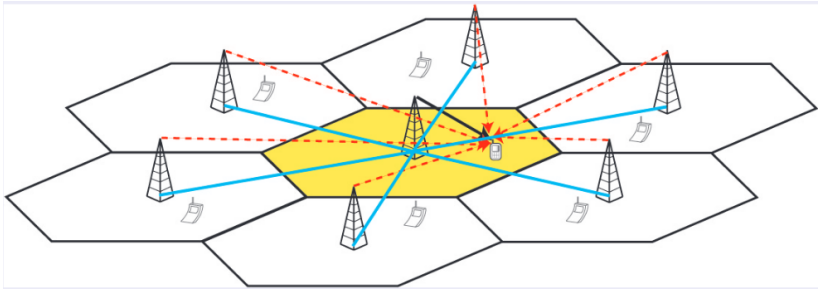
Considérons la gestion d'un grand projet. Il est constitué de différentes étapes à réaliser. Il est logique de penser que certaines tâches doivent être effectuées avant d'autres alors que certaines peuvent très bien être effectuées en même temps. Ainsi, on établit une certaine relation d'ordre entre les étapes. Un premier problème consiste à trouver une planification des tâches qui aboutisse à la réalisation du projet en un minimum de temps. Ensuite, il peut être intéressant de détecter les étapes dites "critiques" dont le moindre retard peut affecter toute la suite du projet.



Exemples de Pb modélisés par la théorie des graphes

Allocation de fréquences dans les réseaux GSM :

Attribuer aux antennes relais des bandes de fréquences pour communiquer avec les usagers.



Sommets : les antennes relais

Arêtes : entre deux antennes trop proches géographiquement l'une de l'autre (niveau d'interférence trop important)

Couleurs : les canaux de fréquences radio

Minimiser le nombre de fréquences utilisées ou pour un nombre de fréquences donné minimiser les interférences

Graphe orienté

Définition

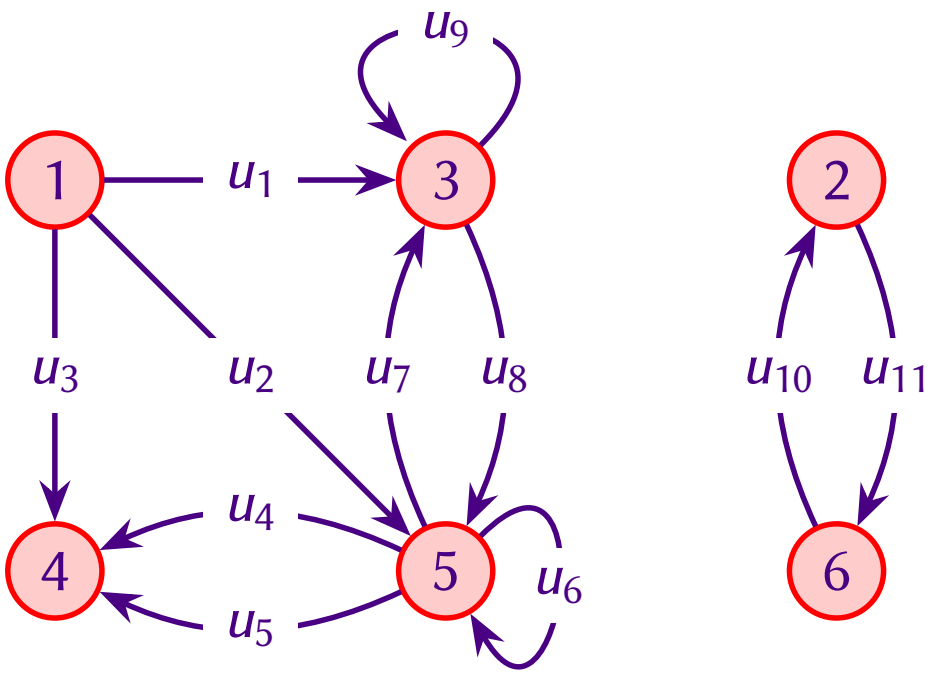
On appelle graphe le couple $G = (X; U)$ formé de deux ensembles : un ensemble X dont les éléments sont appelés sommets, et un ensemble U qui est une famille d'éléments de $X \times X$, dont les éléments sont appelés arcs.

Un graphe orienté est généralement déterminé par la donnée de :

1. un ensemble fini $X = \{x_1, x_2, ..., x_n\}$ d'éléments appelés sommets. Si X possède n éléments distincts, le graphe sera dit d'ordre n .
2. un ensemble fini $U = \{u_1, u_2, ..., u_m\}$ dont les éléments sont des couples ordonnés de sommets appelés arcs. U est donc une famille d'éléments du produit cartésien $X \times X$.

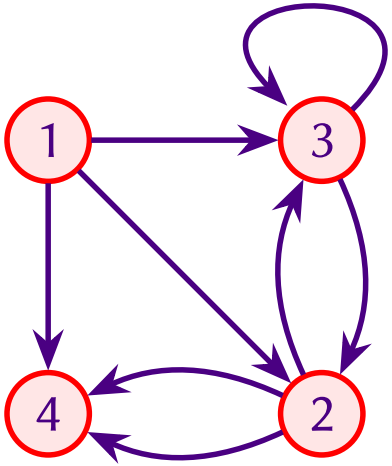
Graphe orienté

Un exemple de graphe d'ordre 6 :

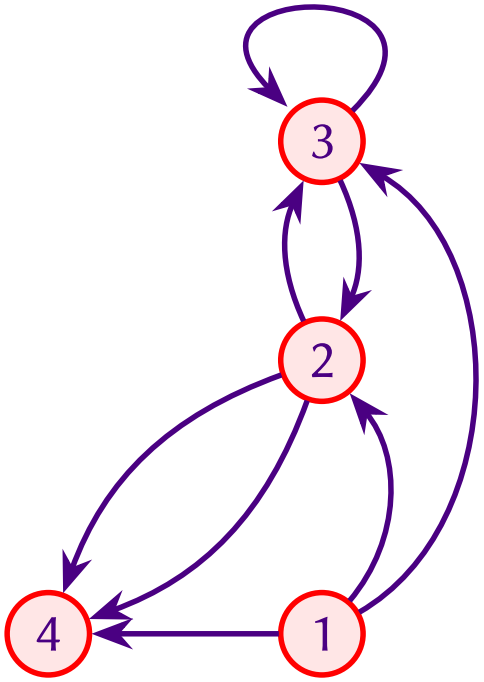


$X = \{1, 2, 3, 4, 5, 6\}$ et
$U = \{u_1 = (1, 3)\,; u_2 = (1, 5)\,; u_3 = (1, 4)\,; \ldots; u_{11} = (2, 6)\}$.

Graphe orienté



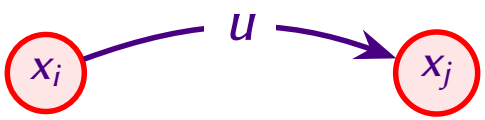
Graphe_1



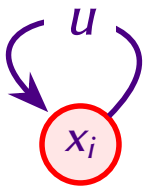
Graphe_2

Ne vous fiez pas à l'aspect visuel de deux graphes pour les comparer.
Seule la comparaison des ensembles de sommets et d'arcs est fiable !

Extrémités d'un arc. Pour un arc $u = (x_i, x_j)$, x_i est l'extrémité initiale, x_j l'extrémité finale (ou bien origine et destination). L'arc u part de x_i et arrive à x_j .



Boucle. On appelle boucle un arc dont l'extrémité initiale est égale à son extrémité finale.



Graphe orienté

Successeur. On dit que x_j est successeur de x_i si $(x_i, x_j) \in U$ (c-à-d. s'il existe un arc ayant x_i comme extrémité initiale et x_j comme extrémité finale). L'ensemble des successeurs de x_i est noté $\Gamma^+(x_i)$.

$$\Gamma^+(x_i) = \{x_j \in X \mid u = (x_i, x_j) \in U\}$$

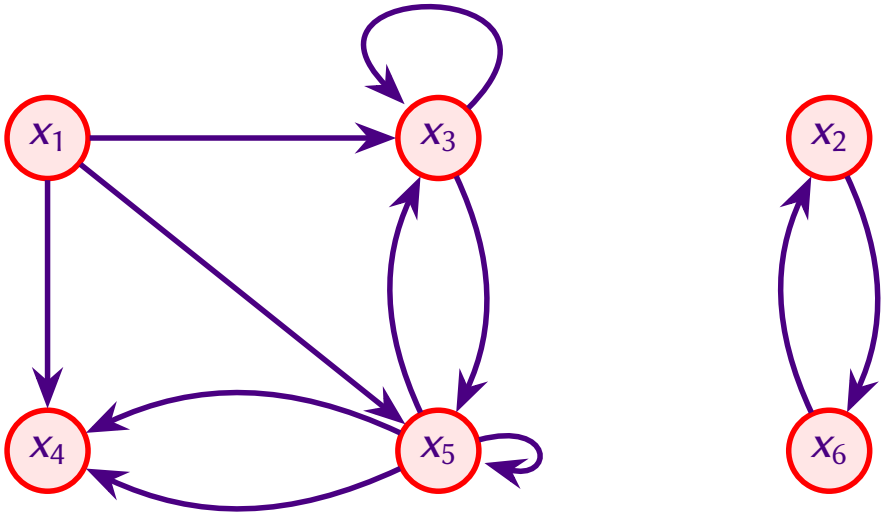
Prédécesseur. On dit que x_j est prédécesseur de x_i si $(x_j, x_i) \in U$ (c-à-d. s'il existe un arc ayant x_j comme extrémité initiale et x_i comme extrémité finale). L'ensemble des prédécesseurs de x_i est noté $\Gamma^-(x_i)$.

$$\Gamma^-(x_i) = \{x_j \in X \mid u = (x_j, x_i) \in U\}$$

Sommets adjacents. On dit que x_i est adjacent à x_j s'il est prédécesseur ou successeur de x_j . Ces sommets peuvent aussi être dits voisins. L'ensemble des voisins de x_i est noté $\Gamma(x)$.

$$\Gamma(x_i) = \Gamma^+(x_i) \cup \Gamma^-(x_i)$$

Graphe orienté



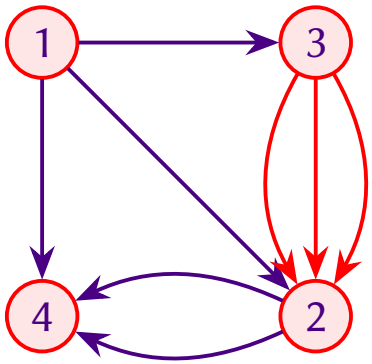
Ainsi sur ce graphe on voit que : $X = \{x_1, x_2, x_3, x_4, x_5, x_6\}$

$$\begin{array}{lll}
\Gamma^+(x_1) = \{x_3, x_4, x_5\} & \Gamma^-(x_1) = \{\emptyset\} & \Gamma(x_1) = \{x_3, x_4, x_5\} \\
\Gamma^+(x_2) = \{x_6\} & \Gamma^-(x_2) = \{x_6\} & \Gamma(x_2) = \{x_6\} \\
\vdots & \vdots & \vdots \\
\Gamma^+(x_5) = \{x_3, x_4, x_5\} & \Gamma^-(x_5) = \{x_1, x_3, x_5\} & \Gamma(x_5) = \{x_1, x_3, x_4, x_5\} \\
\Gamma^+(x_6) = \{x_2\} & \Gamma^-(x_6) = \{x_2\} & \Gamma(x_6) = \{x_2\}
\end{array}$$

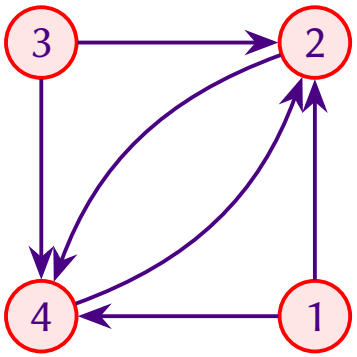
Degré. Soit x_i un sommet d'un graphe G .

- Le demi-degré extérieur de x_i , noté $d^+(x_i)$, est le nombre d'arcs ayant x_i comme extrémité initiale.
- Le demi-degré intérieur de x_i , noté $d^-(x_i)$, est le nombre d'arcs ayant x_i comme extrémité finale.
- Le degré de x_i est $d(x_i) = d^+(x_i) + d^-(x_i)$.

p-graphe. Dans la séquence U, il peut y avoir plusieurs couples identiques. Un p-graphe est un graphe dans lequel il n'existe jamais plus de p arcs de la forme (x_i, x_j) entre deux sommets quelconques.



3-Graphe

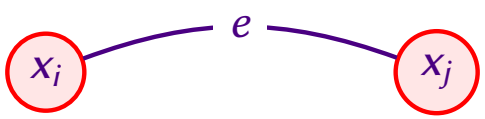


1-Graphe

Remarque : dans le cas d'un 1-graphe, on a $d^+(x) = |\Gamma^+(x_i)|$ et $d^-(x_i) = |\Gamma^-(x_i)|$.

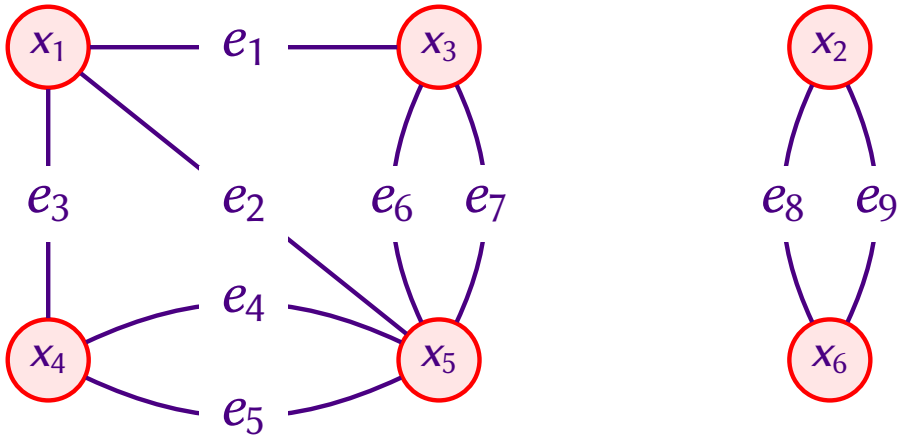
Graphe non orienté

Lors de l'étude de certaines propriétés, il arrive que l'orientation des arcs ne joue aucun rôle. On s'intéresse simplement à l'existence de relations entre deux sommets. Une ligne continue sans orientation, joignant x_i et x_j, est appelé arête qu'on note $e = [x_i, x_j]$.



Graphe non orienté

Si U est l'ensemble des arêtes, on parle alors du multi-graphe $G = (X, U)$. Un multi-graphe est par ailleurs appelé un graphe simple s'il n'a pas de boucle et s'il existe au maximum une arête entre deux sommets.



Pour une arête $e = [x_i, x_j]$, on dit que e est incidente aux sommets x_i et x_j . Le degré d'un sommet est le nombre d'arêtes qui lui sont incidentes.

Matrice d'adjacence d'un 1-graphe

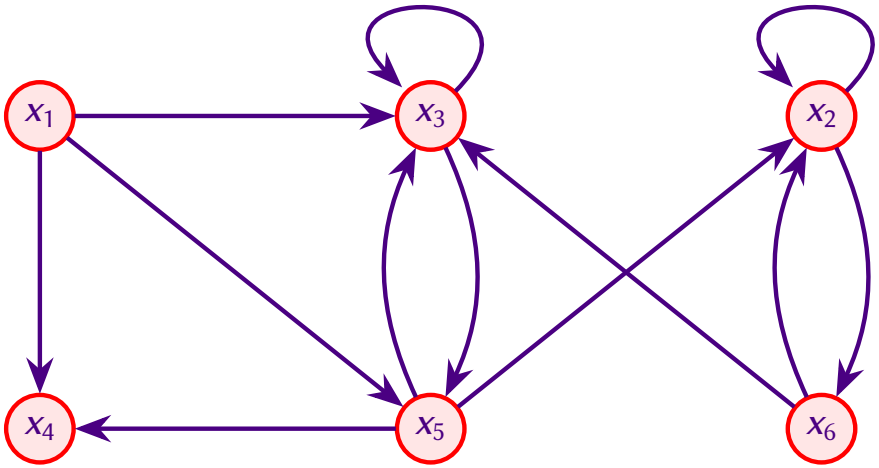
Un 1-graphe G peut être définit par le couple (X, U). Il est aussi déterminé par la donnée de X et l'application Γ^+ (ou Γ^-). Il peut enfin être caractérisé par sa matrice d'adjacence.

La matrice d'adjacence de G est la matrice $n \times n$,

$$M(G) = (m_{ij}) \text{ où } m_{ij} = \begin{cases} 1 & \text{si } (x_i, x_j) \in U \\ 0 & \text{si } (x_i, x_j) \notin U \end{cases}$$

Matrice d'adjacence d'un 1-graphe

Exemple :



$$M(G) = \begin{bmatrix} 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Matrice d'adjacence d'un 1-graphe

Voici quelques propriétés évidentes de la matrice d'adjacence :

- $\forall i, d^{+}(x_i) = \sum_{j=1}^{n} m_{ij}$ (la somme de la i ligne).
- $\forall j, d^{-}(x_j) = \sum_{i=1}^{n} m_{ij}$ (la somme de la j colonne).
- $\sum_{i=1}^{n}\sum_{j=1}^{n} m_{ij} = \mid U \mid$.
- G est symétrique $\Leftrightarrow M(G)$ est une matrice symétrique $(\forall i, j \quad m_{ij} = m_{ji})$.
- Le nombre de boucles de G est la trace de $M(G)$ $\left(tr(M) = \sum_{i=1}^{n} m_{ii}\right)$.

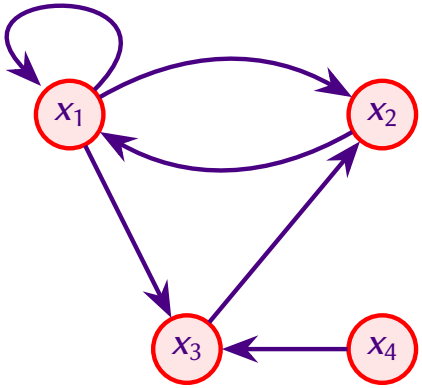
Existence d'un chemin de longueur donnée

Théorème :

Soit G un 1-graphe d'ordre n, où $X = \{x_1, x_2, ..., x_n\}$ et soit M sa matrice d'adjacence associée. La matrice $M^k = \underbrace{M \times M \times \cdots \times M}_{\text{k fois}}$, avec $(k \leq n)$, alors m_{ij}^k est le nombre de chemin de longueur k du sommet x_i au sommet x_j.

Existence d'un chemin de longueur donnée

Exemple. Considérons le graphe orienté de la figure suivante :



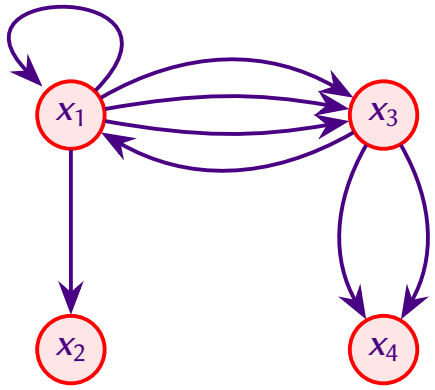
$$M = \begin{bmatrix} 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \quad M^2 = \begin{bmatrix} 2 & 2 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

$$M^3 = \begin{bmatrix} 4 & 3 & 2 & 0 \\ 2 & 2 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

Par exemple, $m_{12}^{(3)} = 3$ nous dit qu'il y a 3 chemins de longueur 3 de x_1 à x_2.
Ce sont $[x_1, x_1, x_1, x_2]$, $[x_1, x_2, x_1, x_2]$, $[x_1, x_1, x_3, x_2]$.

Matrice d'adjacence d'un multi-graphe

Soit le multi-graphe suivant :



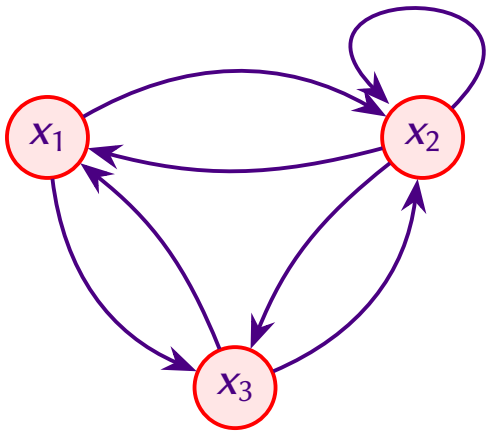
La matrice d'adjacence du multi-graphe orienté G prend alors ses valeurs dans $\mathbb{N}$.

$$M = \begin{bmatrix} 1 & 1 & 3 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 2 \\ 0 & 0 & 0 & 0 \end{bmatrix}, M^3 = \begin{bmatrix} 20 & 7 & 21 & 12 \\ 0 & 0 & 0 & 0 \\ 7 & 2 & 6 & 6 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

. il y a, par exemple 21 chemins de longueur 3 de x_1 à x_3.

Exercice

Exercice. Trouver le nombre de circuits de longueur 4 dans le graphe suivant :



Solution

Soit M la matrice d'adjacence. On à $M = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 0 \end{bmatrix}$, $M^4 = \begin{bmatrix} 9 & 12 & 8 \\ 12 & 17 & 12 \\ 8 & 12 & 9 \end{bmatrix}$

Comme $tr(M^4) = 9 + 17 + 9 = 35$, il y a 35 circuits de longueur 4 dans G.

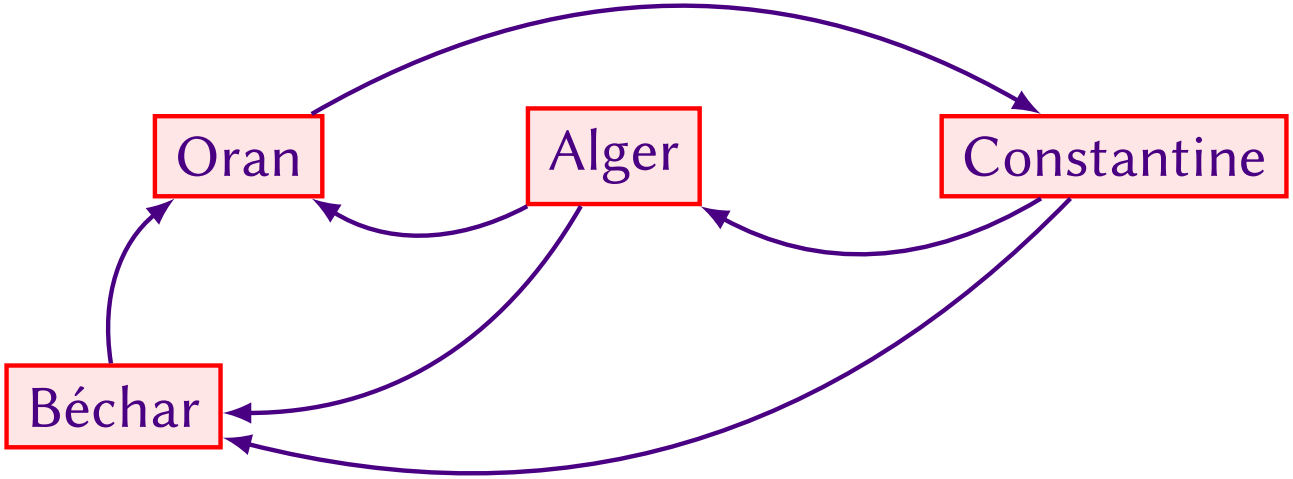
Exercice

Exercice. On considère quatre villes Alger, Oran, Constantine, Béchar où le trafic aérien est encore très réduit : il existe seulement un vol direct d'Alger vers Oran et vers Béchar, d'Oran vers Constantine, de Constantine vers Alger et vers Béchar, de Béchar vers Oran.

1. Représenter les données par un graphe convenable.
2. Vérifier qu'il existe au moins un vol de départ de chaque ville vers chaque ville de destination.

Solution

1.



2. La matrice $M(G)$ associée à ce graphe est

$$M(G) = \begin{array}{c} \\ \text{ALG} \\ \text{ORA} \\ \text{CZL} \\ \text{BCH} \end{array} \begin{array}{c} \begin{array}{cccc} \text{ALG} & \text{ORA} & \text{CZL} & \text{BCH} \end{array} \\ \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix} \end{array}$$

Solution

On calcule $M^2(G) = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 2 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$ et $M^3(G) = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 0 & 2 & 0 & 1 \\ 0 & 1 & 2 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}$.

On calcule $M(G) + M^2(G) + M^3(G) = \begin{bmatrix} 1 & 2 & 2 & 2 \\ 1 & 2 & 1 & 2 \\ 1 & 3 & 2 & 2 \\ 1 & 1 & 1 & 1 \end{bmatrix}$

Cette dernière matrice ne comportant pas de 0, et ne comportant que des entiers inférieurs ou égaux à 3, il existe toujours une chaîne de longueur au plus égale à 3 entre deux aéroports, c'est-à-dire un voyage comportant au plus deux escales.

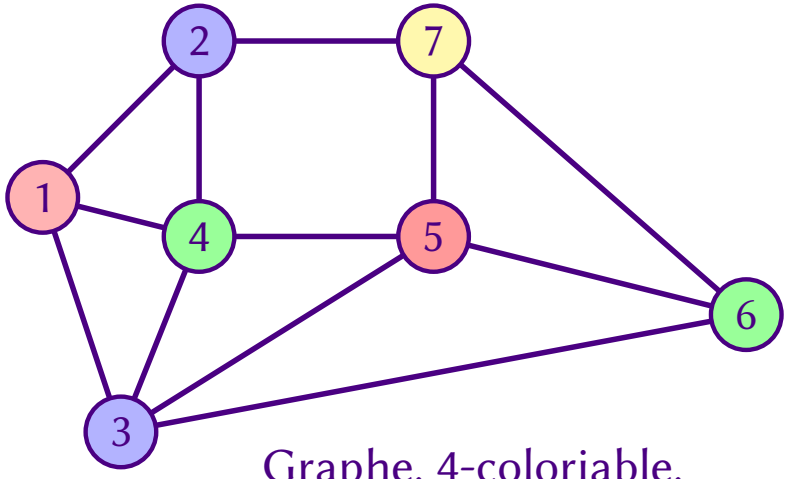
coloration d'un graphe

C'est une notion non orientée. On définit deux types de coloration pour un graphe $G = (X, U)$: coloration des sommets et coloration des arêtes.

La coloration des sommets (resp. arêtes) d'un graphe G correspond à l'affectation d'une couleur à chacun de ses sommets (resp. arêtes) de telle sorte que deux sommets (resp. arêtes) adjacents ne soient pas porteurs de la même couleur.

On cherche donc, à obtenir une coloration des sommets d'un graphe qui respecte la contrainte suivante : deux sommets voisins n'ont jamais la même couleur. De plus,On cherche à minimiser le nombre de couleurs utilisées.

Définition : Un graphe est dit p-chromatique si ses sommets admettent une coloration en p couleurs. On appellera nombre chromatique $\chi(G)$ (resp. indice chromatique $q(G)$) le nombre minimum de couleurs distinctes nécessaires pour effectuer une coloration des sommets (resp. arêtes) du graphe G.



Graphe. 4-coloriable.

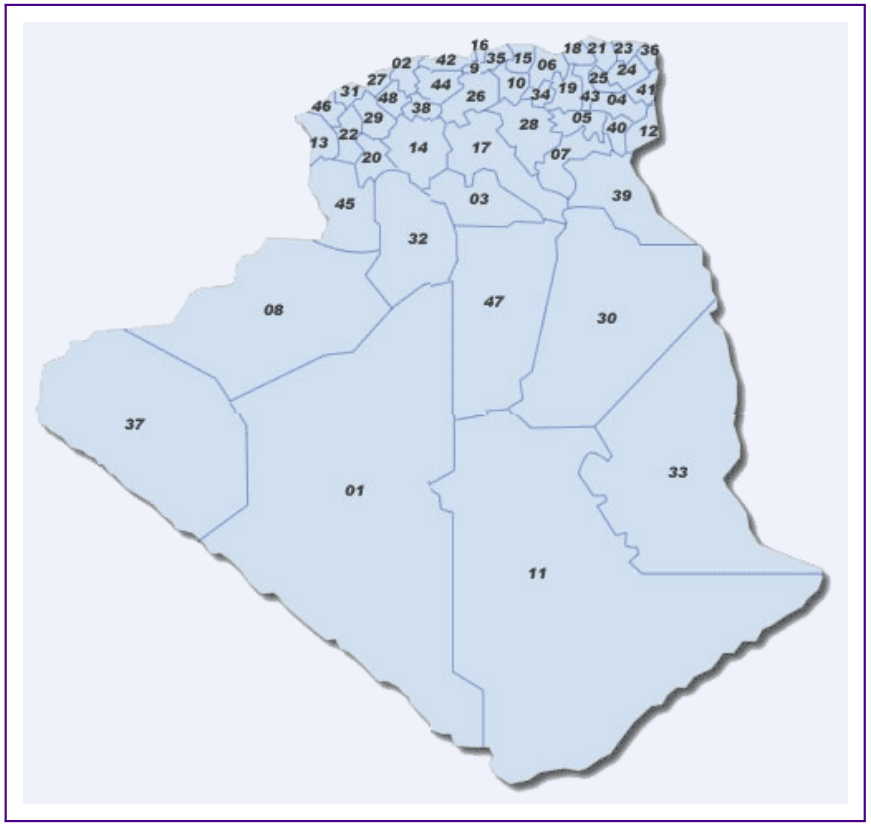
coloration d'un graphe

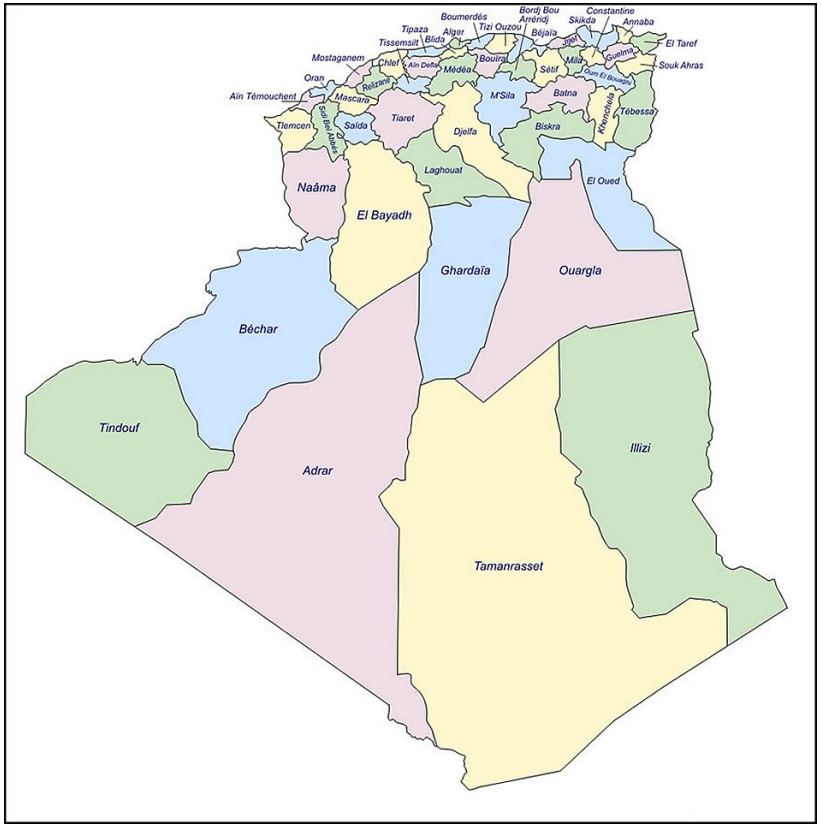
On appelle « K-coloration » d'un graphe $G = (X, U)$ une partition de l'ensemble des sommets de X de G en K_classes $(X_1, X_2, ..., X_k)$, de telle façon que deux sommets d'une même classe ne soient pas adjacents, et les sommets d'une classe sont coloriés de la même couleur. Autrement dit, deux sommets adjacents n'ont pas la même couleur.

Un très grand nombre de problèmes concrets peuvent se modéliser en termes de coloration d'un graphe. Donnons quelques exemples :

Problème de coloration d'une carte géographique

Est-il il possible de colorier toute carte géographique avec 4 couleurs, de sorte que deux régions ayant une frontière commune soient de couleurs différentes ?





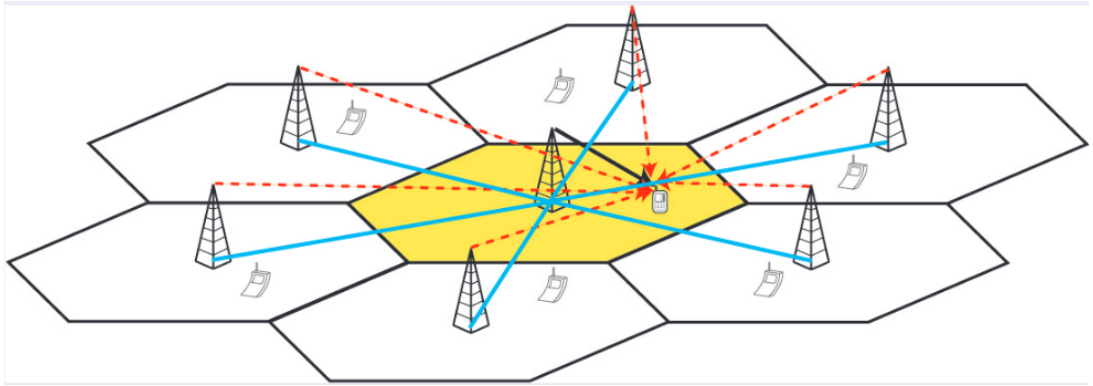
Problème de coloration d'une carte géographique

Ce problème célèbre, longtemps resté ouvert, n'a été résolu qu'en 1976 (Kenneth Appel et Wolfgang Haken); La démonstration fit grand bruit car c'est le premier théorème de l'histoire des mathématiques qui a nécessité l'usage systématique de l'ordinateur. Il s'énonce aujourd'hui comme le « théorème des quatre couleurs » : tout graphe planaire (sans boucles) est 4-chromatique.
En effet, à toute carte de géographie, on peut associer le graphe suivant :

Sommets : les régions de la carte
Arêtes : entre deux regions si et seulement si les deux régions ont une frontière commune.

Une coloration des sommets du graphe est alors équivalente à une coloration des régions.

Problème d'Allocation de Fréquence



Avec le développement des réseaux de télécommunication modernes, on assiste au déploiement d'un nombre sans cesse croissant d'antennes-relai à travers le pays. Il est donc important de pouvoir assurer le transport de l'information entre les différentes antennes, afin de satisfaire aux différents acteurs du marché : les opérateurs doivent être en mesure de transmettre les communications de leurs clients respectifs, sans se gêner entre eux, le tout en minimisant le coût de ces opérations.

Problème d'Allocation de Fréquence

Ce problème est généralement qualifié de « Problème d'Allocation de Fréquence » : étant donné un réseau d'antenne, il faut pouvoir allouer différentes fréquences aux antennes émettrices et réceptrices de sorte qu'elles puissent véhiculer de l'information sans interférer. Il est possible de résoudre le problème grâce au formalisme de la théorie des graphes.

Sommets : les antennes relais
Arêtes : entre deux antennes trop proches géographiquement l'une de l'autre (niveau d'interférence trop important)
Couleurs : les canaux de fréquences radio.

Minimiser le nombre de fréquences utilisées ou pour un nombre de fréquences donné minimiser les interférences

Problème de l'emploi du temps

Allocation de créneaux horaires à des événements : cours, examens...

En fin d'année universitaire, on désire faire passer simultanément des examens écrits dans un laps de temps minimum sans créer d'impossibilité pour les étudiants.

On désire donc que chaque étudiant ait au plus un examen par jour, que tous les étudiants inscrits à un même examen le passent en même temps et de plus on veut rendre minimum le nombre de jours nécessaires pour terminer les examens.

Sommets : les examens
Arêtes : deux examens ayant des étudiants communs
Couleurs : les créneaux horaires.

Minimiser la durée totale des événements

Problème de l'emploi du temps

Exemple : Cinq étudiants A, B, C, D, E doivent passer les examens suivants.

A	Algorithmique, Anglais, Logique
B	Probabilités, Compilation
C	Réseau, Logique
D	Anglais, Probabilités, Réseau
E	Algorithmique, Compilation

Sachant que chaque étudiant ne peut se présenter qu'à une épreuve par jour, quel est le nombre minimal de jours nécessaire à l'organisation de toutes les épreuves ? (Réponse 3).

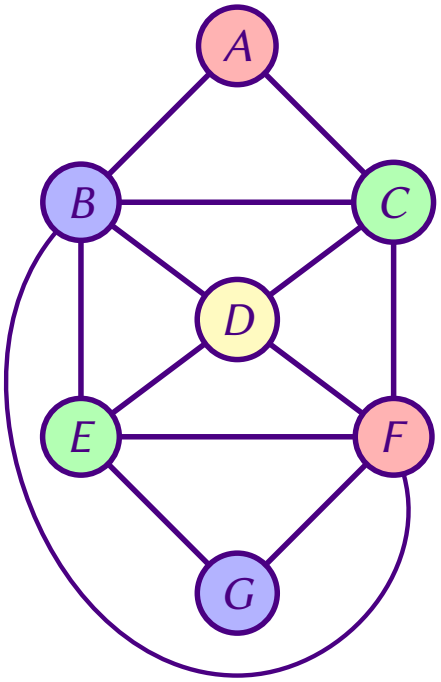
Algorithme de Welsh et Powell

Input : Un graphe G non orienté ;
Input : L = liste des sommets dans l'ordre décroissant de leur degré ;
Output : Coloration des sommets du graphe G ;

```
Couleur_courante ← 0;
Tantque (L ≠ ∅) Faire
    Couleur_courante ← Couleur_courante + 1;
    Colorier (s) le premier sommet de (L) avec (Couleur_courante);
    éliminer (s) de (L);
    V = liste des voisins de (s);
    Pour (x ∈ L) Faire
        Si (x ∉ V) Alors
            colorier le sommet (x) avec (Couleur_courante);
            ajouter les voisins de (x) à (V);
            éliminer le sommet (x) de (L);
        fin Si
    fin Pour
fin Tantque
```

coloration d'un graphe

Sommets	B	F	C	D	E	A	G
Degrés	5	5	4	4	4	2	2
Bleu	x						x
Rouge		x				x	
Vert			x		x		
Jaune				x			



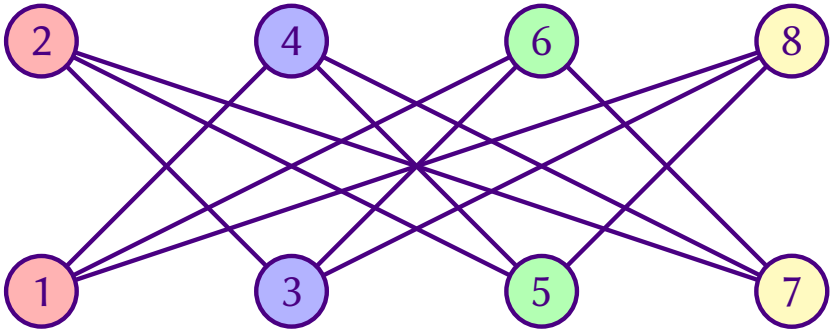
$C_1 = \{B, G\}; C_2 = \{A, F\}; C_3 = \{C, E\}; C_4 = \{D\}$

Remarque. La coloration n'est pas forcément unique. Par exemple, $C1 = \{A, D, G\}; C2 = \{C, E\}; C3 = \{B\}; C4 = \{F\}$.

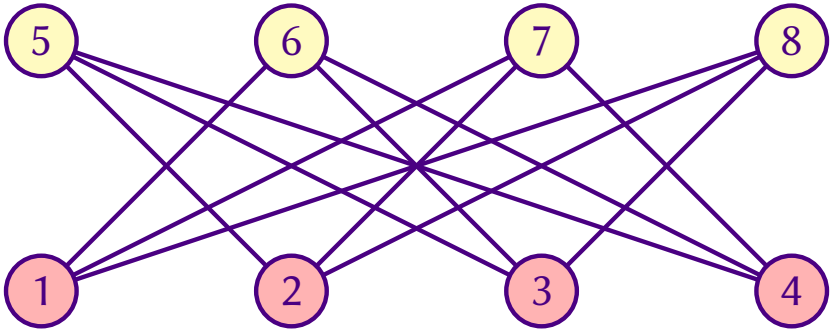
Un sous-graphe est stable si ses sommets ne sont reliés par aucune arête. Une coloration avec k couleurs est donc une partition de l'ensemble des sommets en k sous graphes stables.

L'algorithme de coloration de Welsh et Powell proposé ne donne pas nécessairement le nombre minimum de couleurs. Mais peut être très mauvais du point de vue du nombre de couleurs utilisées.

Exemple avec les graphes construits comme suivant :



C1={1,2}; C2={3,4}; C3={5,6}; C4={7,8}



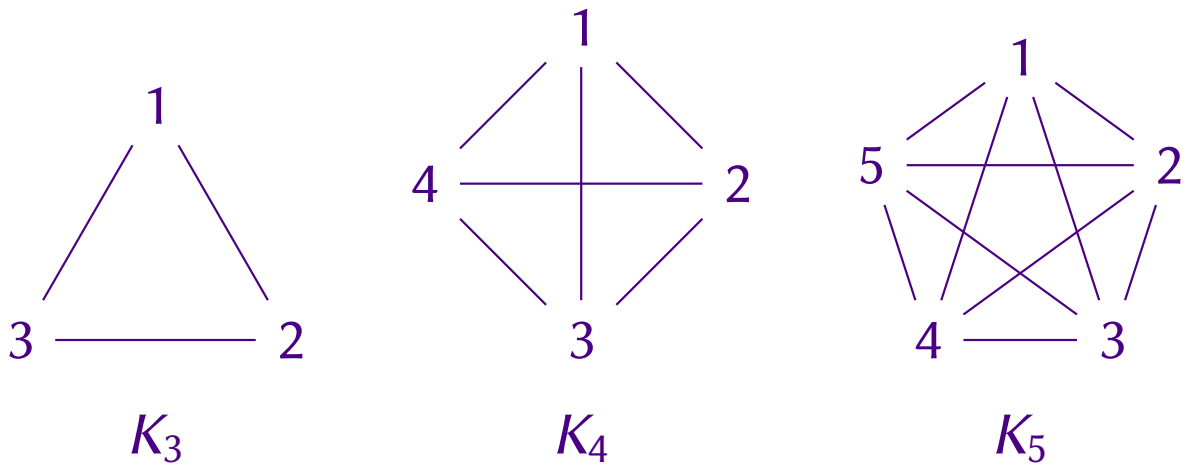
C1={1,2,3,4}; C2={5,6,7,8}

Encadrement du nombre chromatique

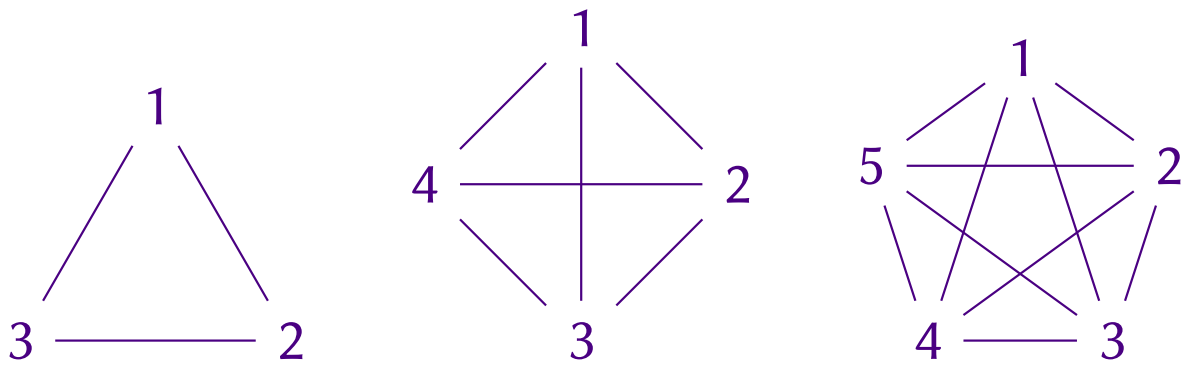
Le nombre chromatique noté $\chi(G)$ d'un graphe est le plus petit nombre de couleurs nécessaires pour colorier les sommets d'un graphe, de sorte que deux sommets adjacents ne soient pas de la même couleur.

Il n'existe pas de « formule » ou d'algorithme efficace permettant de donner le nombre chromatique d'un graphe mais deux résultats permettent d'encadrer le nombre chromatique d'un graphe.

Définition. **(graphe complet)** : On appelle « graphe complet à n sommets », souvent noté K_n, le graphe d'ordre n ayant le plus d'arcs/arêtes possibles. Un graphe simple est dit complet si tous ses sommets sont adjacents, c'est à dire si toutes les arêtes possibles existent.

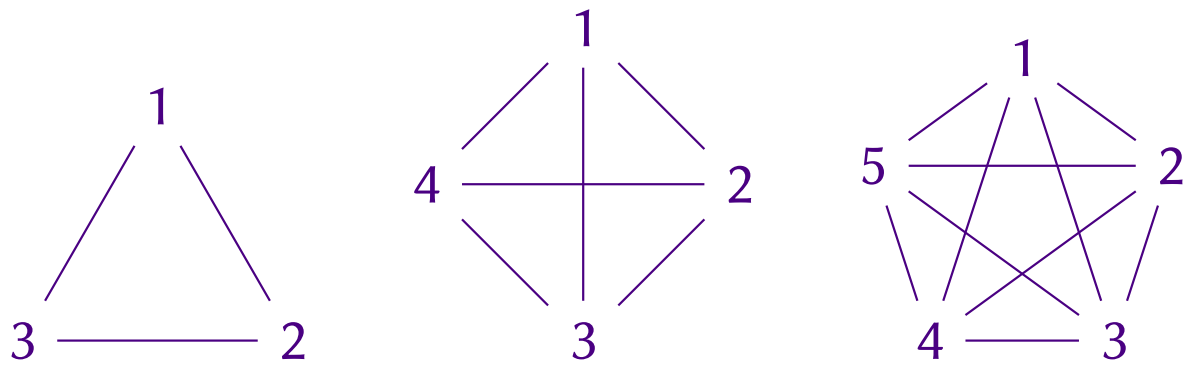


Encadrement du nombre chromatique



Théorème. Pour le graphe complet K_n le nombre chromatique est n.

Encadrement du nombre chromatique



Théorème. Pour le graphe complet K_n le nombre chromatique est n.

Démonstration. Comme un sommet donné est adjacent aux $(n-1)$ autres sommets, il faut au moins n couleurs pour obtenir une coloration valide de K_n.
Avec n couleurs (une pour chaque sommet), on obtient une coloration de K_n et $\chi(K_n) = n$. $\square$

Encadrement du nombre chromatique

Théorème. Si $\Delta(G)$ est le plus grand degré des sommets du graphe G alors le nombre chromatique est inférieur ou égal à $\Delta(G) + 1$.
Si m est l'ordre du plus grand des sous-graphes complets du graphe G alors le nombre chromatique est supérieur ou égal à m. Ainsi on a :

$$m \leq \chi(G) \leq \Delta(G) + 1$$

Encadrement du nombre chromatique

Théorème. Si $\Delta(G)$ est le plus grand degré des sommets du graphe G alors le nombre chromatique est inférieur ou égal à $\Delta(G) + 1$.
Si m est l'ordre du plus grand des sous-graphes complets du graphe G alors le nombre chromatique est supérieur ou égal à m. Ainsi on a :

$$m \leq \chi(G) \leq \Delta(G) + 1$$

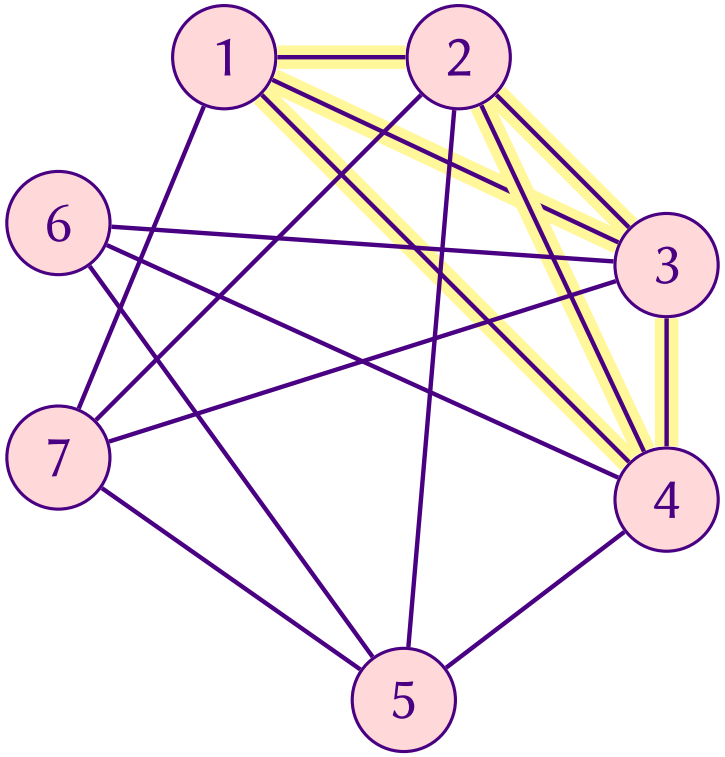
Démonstration. Soit $\Delta(G)$ le degré maximum des sommets d'un graphe G. Considérons une liste de $(\Delta(G)+1)$ couleurs. Chaque sommet x du graphe est adjacent à $\Delta(G)$ sommets au plus, ses voisins utilisent au maximum $\Delta(G)$ couleurs, le nombre de couleurs déjà utilisées pour colorer ces sommets est donc inférieur ou égal à $\Delta(G)$.
Il reste donc au moins une couleur non utilisée dans la liste de couleurs, avec laquelle nous pouvons colorer le sommet x . Ainsi, $\chi(G) \leq \Delta(G) + 1$.
Soit H un sous graphe de G . Si une coloration de H nécessite m couleurs, il en va au moins de même pour le graphe G et $\chi(G) \geq m$. $\square$

Planing des examens :

Une université doit organiser les horaires des examens. On suppose qu'il y a 7 épreuves à planifier, correspondant aux cours numérotés de 1 à 7 et que les paires de cours suivantes ont des étudiants communs : 1 et 2, 1 et 3, 1 et 4, 1 et 7, 2 et 3, 2 et 4, 2 et 5, 2 et 7, 3 et 4, 3 et 6, 3 et 7, 4 et 5, 4 et 6, 5 et 6, 5 et 7 et 6 et 7. Chaque candidat ne passe qu'une épreuve par jour. Comment organiser ces épreuves sur une durée miminale.

Exemple

Construisons le graphe G dont les sommets sont les épreuves numérotées de 1 à 7, une arête relie deux de ses sommets lorsque les deux cours correspondant possèdent des étudiants communs :



Exemple

Planifier les examens en un temps minimal consiste à déterminer une k_coloration de G; avec $k = \chi(G)$.
G possède un sous-graphe complet d'ordre 4 (de sommets 1, 2, 3, 4), donc

$$\chi(G) \geq 4$$

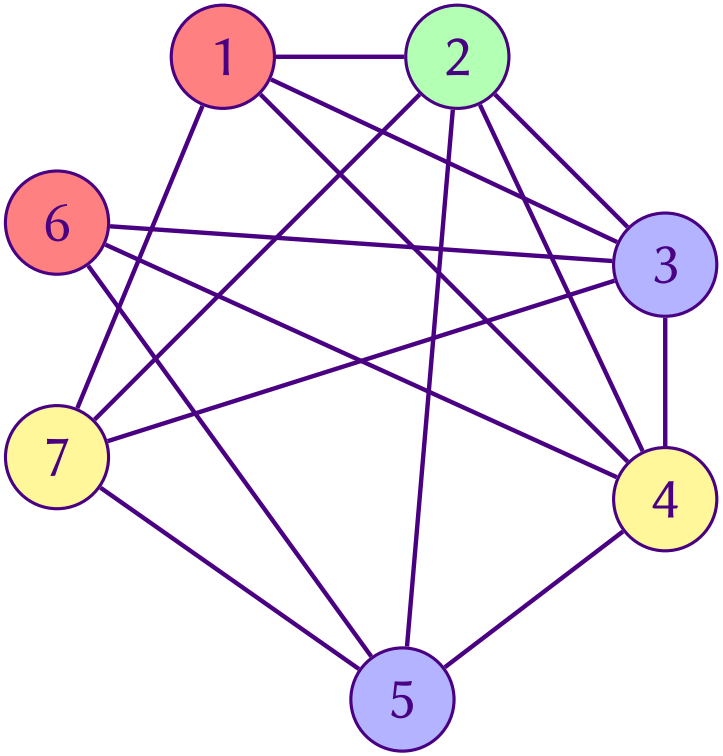
Déterminons une partition des sommets de G en sous-ensembles stables.

$$S_1 = \{1, 6\}$$
$$S_2 = \{2\}$$
$$S_3 = \{3, 5\}$$
$$S_4 = \{4, 7\}$$

d'où $\chi(G) \leq 4$; et finalement $\chi(G) = 4$.

Exemple

Les examens peuvent être répartis en 4 périodes, de la manière suivante :



1 jour, épreuves des cours 1 et 6 ,
2 jour, épreuve du cours 2 ,
3 jour, épreuves des cours 3 et 5 ,
4 jour, épreuves des cours 4 et 7 ,

Décomposition d'un graphe en niveau

Définition du problème.

Etant donné un graphe orienté connexe acyclique (i.e. sans circuit) $G(X, U)$, on veut trouver un ordre linéaire des sommets de G tel que si $(u, v) \in U$ alors u apparaît avant v dans cet ordre. Si le graphe n'est pas acyclique, un tel ordre n'existe pas. Un tel classement est appelé tri topologique.

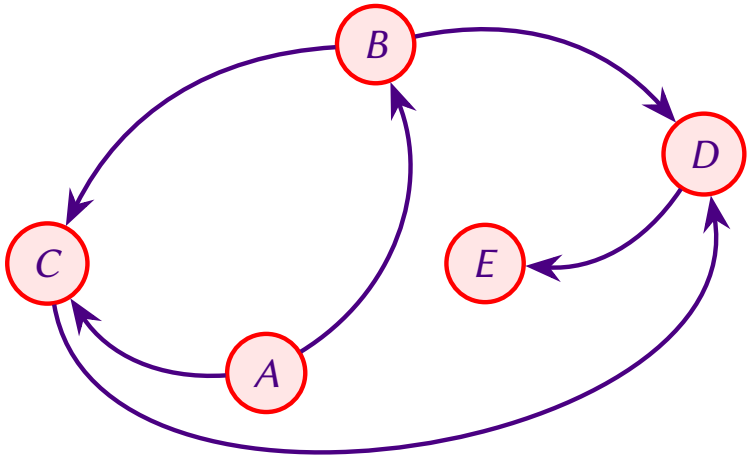
L'intérêt de ce tri est d'accélérer certains algorithmes, en leur faisant traiter tout sommet avant ses successeurs. Par exemple, l'algorithme du plus court chemin dû à Bellman, il est en $O(N.M)$; il devient en $O(M)$ sur un graphe sans circuit préparé par un tri topologique.

Décomposition d'un graphe en niveau

Exemple. Soit le graphe suivant $G(X, U)$ représenter par :

1. Soit le dictionnaire des prédécesseurs des sommets du graphe $G(X, U)$.

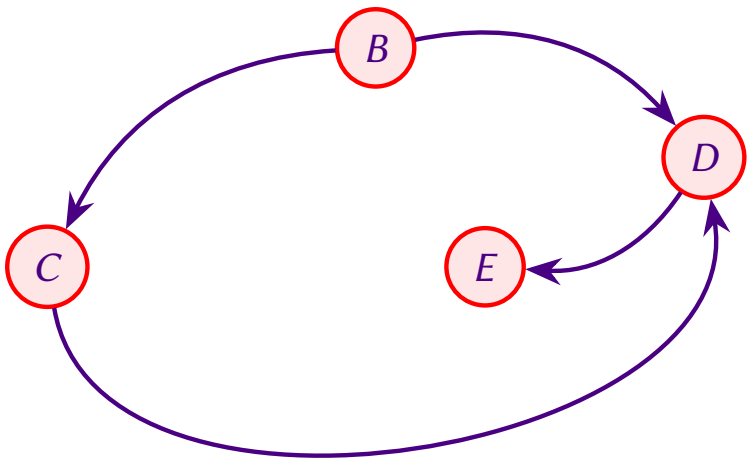
X	$\Gamma^{-}(X)$
A	$\emptyset$
B	A
C	A, B
D	C, B
E	D



On repère dans le dictionnaire des prédécesseurs du graphe les sommets n'ayant pas de prédécesseurs ($\Gamma^{-}(X) = \emptyset$). Dans notre cas, le sommet A n'a pas de prédécesseurs, il est donc de niveau nul $N_0 = \{A\}$.

Décomposition d'un graphe en niveau

2. On barre dans la colonne de $\Gamma^-(X)$ tous les sommets du niveau N_0, on a une nouvelle colonne $\Gamma^-(X)$ avec G_1 le sous-graphe engendré par $X \setminus N_0$.



X	$\Gamma^-(X)$
—	—
B	$\emptyset$
C	$-, B$
D	C, B
E	D

On repère dans le dictionnaire des prédécesseurs du nouveau graphe les sommets n'ayant pas de prédécesseurs $(\Gamma^-(X) = \emptyset)$. Dans notre cas, le sommet B n'a pas de prédécesseurs, il est donc de niveau $N_1 = \{B\}$.

Décomposition d'un graphe en niveau

On continue le même procédé jusqu'à ce qu'on termine tous les sommets du graphe et on représente ainsi le graphe ordonné par niveaux de G.

X	$\Gamma^{-}(X)$
—	—
—	—
C	$\emptyset$
D	$C, -$
E	D

$N_2 = \{C\}$

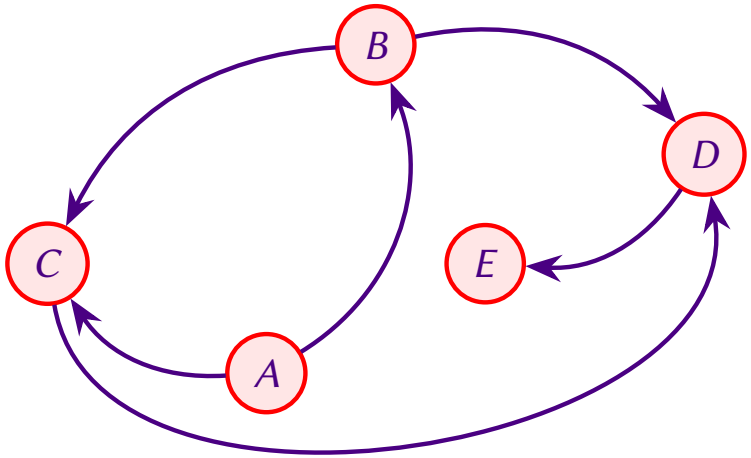
X	$\Gamma^{-}(X)$
—	—
—	—
—	—
D	$\emptyset$
E	D

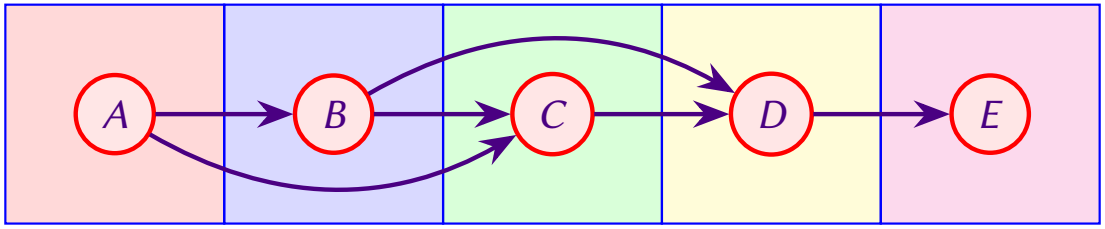
$N_3 = \{D\}$

X	$\Gamma^{-}(X)$
—	—
—	—
—	—
—	—
E	$\emptyset$

$N_4 = \{E\}$

Décomposition d'un graphe en niveau





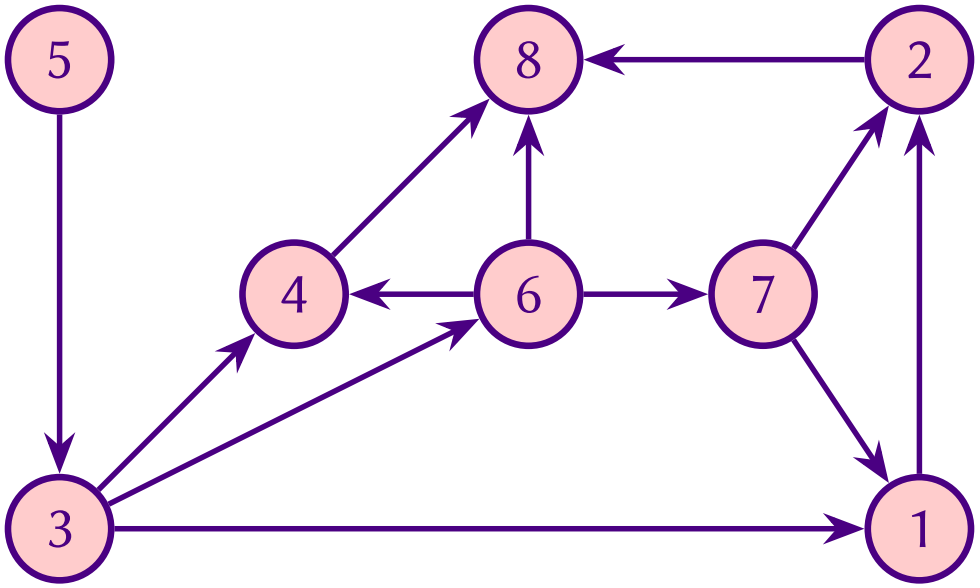
Niveau_0 Niveau_1 Niveau_2 Niveau_3 Niveau_4

Algorithme de décomposition d'un graphe en niveau

Input : Un graphe G orienté acyclique ;
Input : n = nombre de sommets du graphe ;
Input : *niveau* = matrice nulle de taille $1 \times n$;
Input : *degre* = matrice de taille $1 \times n$ contenant les degrés entrants $d^-(x)$;
Output : *niveau* des sommets du graphe G ;

```
Tantque tous les sommets ne sont pas marqués Faire
    L = liste des sommets non-marqués de degré entrant nul ;
    Pour (x ∈ L) Faire
        Pour (y successeur de x dans G) Faire
            Si (niveau(y) < niveau(x) + 1) Alors
                niveau(y) = niveau(x) + 1 ;
                degre(y) = degre(y) − 1 ; //revient à éliminer l'arc (x, y)
            fin Si
        fin Pour
        Marquer le sommet (x) de (L) ;
    fin Pour
fin Tantque
```

Exemple de décomposition d'un graphe en niveau

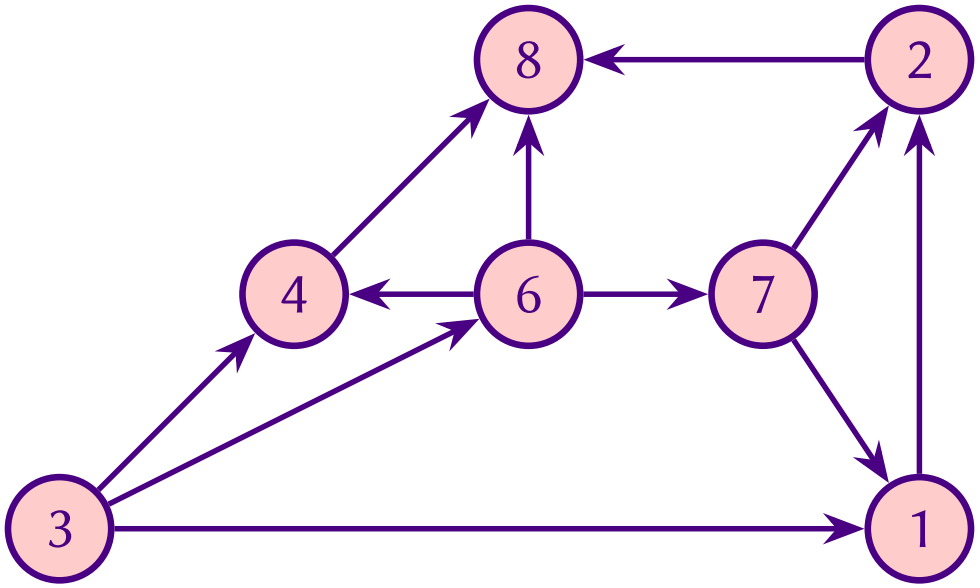


	1	2	3	4	5	6	7	8
niveau =	0	0	0	0	0	0	0	0

$\Uparrow$

$N = 0$

Exemple de décomposition d'un graphe en niveau

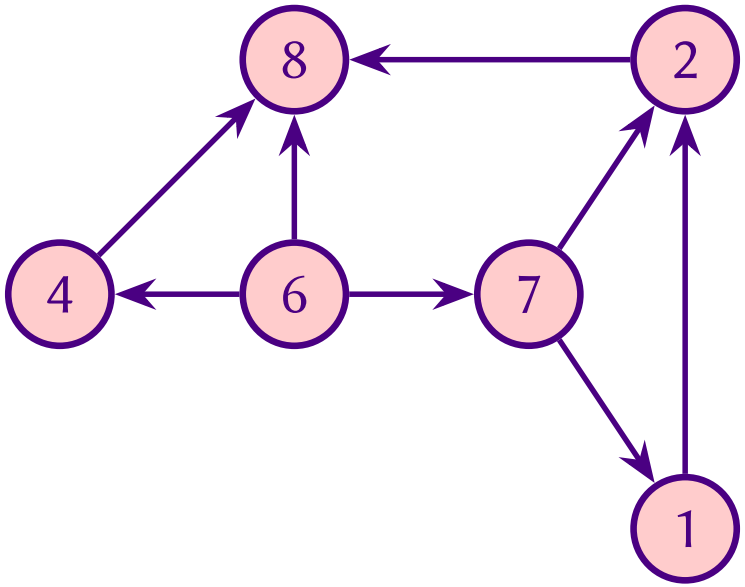


	1	2	3	4	5	6	7	8
niveau =	0	0	1	0	0	0	0	0

$\Uparrow$

$N = 1$

Exemple de décomposition d'un graphe en niveau

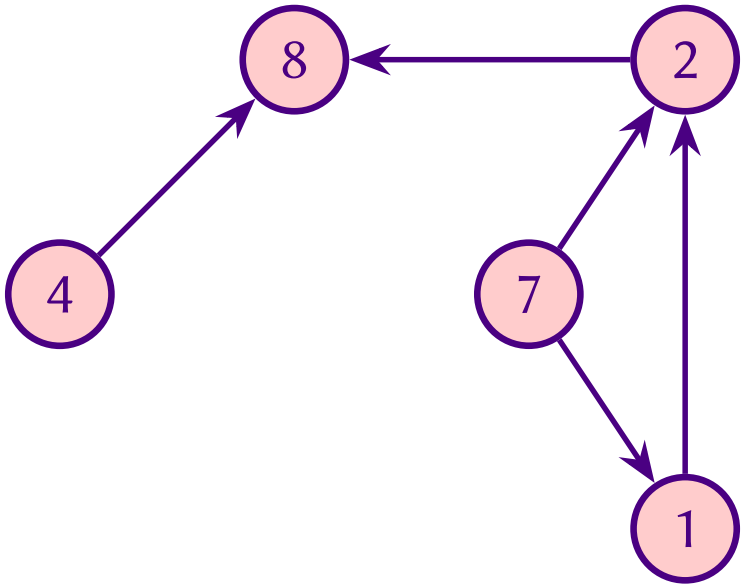


	1	2	3	4	5	6	7	8
niveau =	0	0	1	0	0	2	0	0

$\Uparrow$

$N = 2$

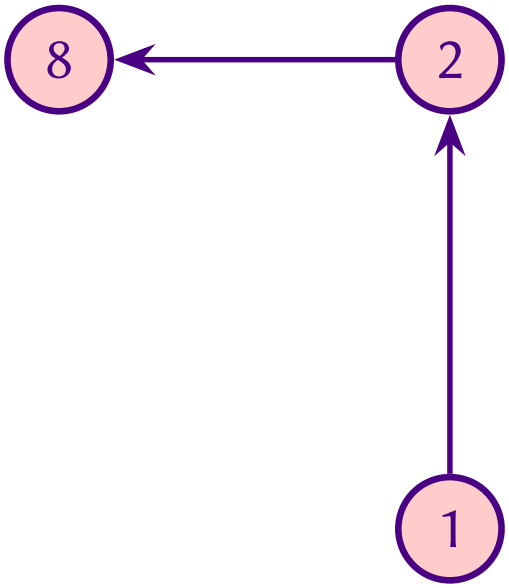
Exemple de décomposition d'un graphe en niveau



	1	2	3	4	5	6	7	8
niveau =	0	0	1	3	0	2	3	0
				⇑			⇑	

$N = 3$

Exemple de décomposition d'un graphe en niveau



	1	2	3	4	5	6	7	8
niveau =	4	0	1	3	0	2	3	0

$\Uparrow$

$N = 4$

Exemple de décomposition d'un graphe en niveau

	1	2	3	4	5	6	7	8
niveau =	4	5	1	3	0	2	3	0

$\Uparrow$

$N = 5$

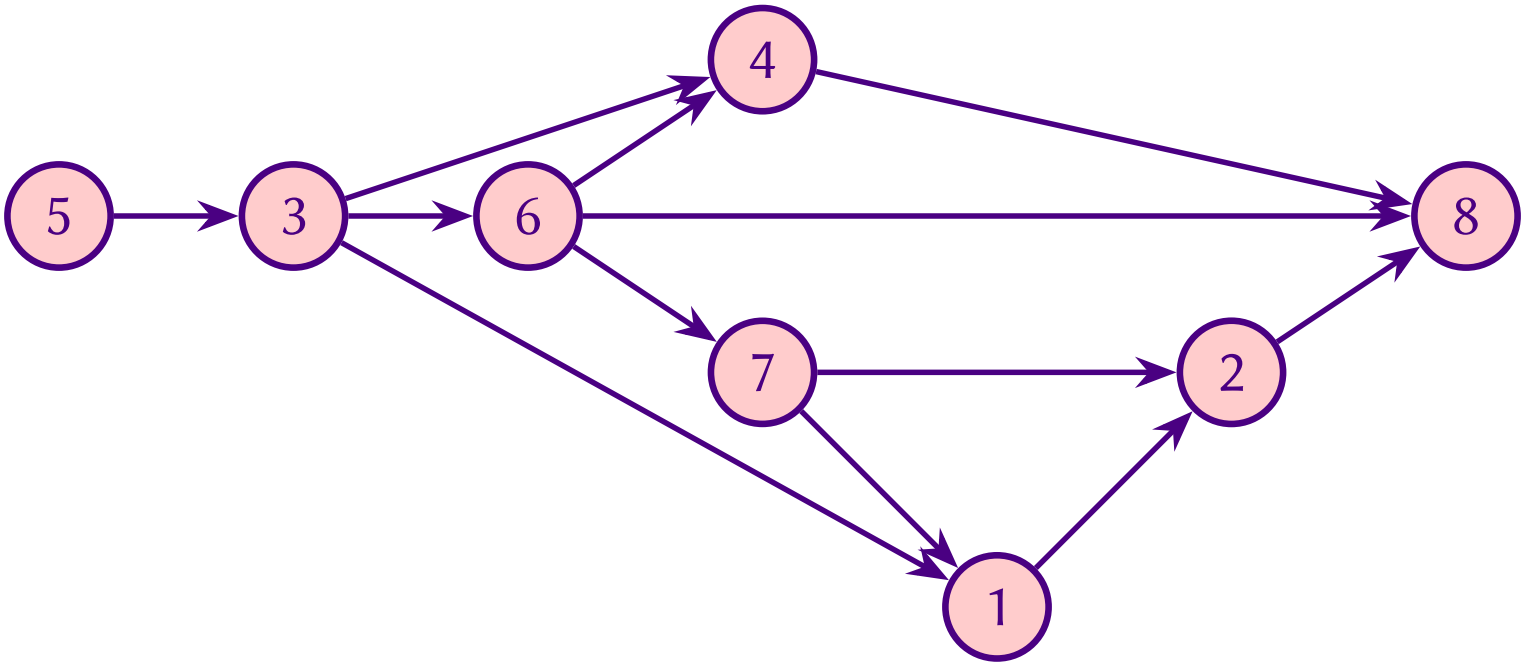
Exemple de décomposition d'un graphe en niveau

	1	2	3	4	5	6	7	8
niveau =	4	5	1	3	0	2	3	6

$\Uparrow$

$N = 6$

Exemple de décomposition d'un graphe en niveau



	1	2	3	4	5	6	7	8
niveau =	4	5	1	3	0	2	3	6

Exercice

Considérons un ordinateur parallèle équipé de plusieurs processeurs, et fonctionnant par cycles d'horloge de durées égales. Au cours d'un cycle d'horloge, chaque processeur peut traiter une opération à un ou deux opérandes et range le résultat dans une variable temporaire. Les variables temporaires, en nombre non limité, on pour noms T_1 , T_2 , ..., etc.

1. On demande en combien de cycles d'horloge au minimum l'expression suivante peut être calculée :

$$\frac{\sin(A^2 + B) - \cos(A^2 + B * C)}{1 + C + A^2}$$

2. Combien de processeurs sont suffisant pour calculer l'expression.

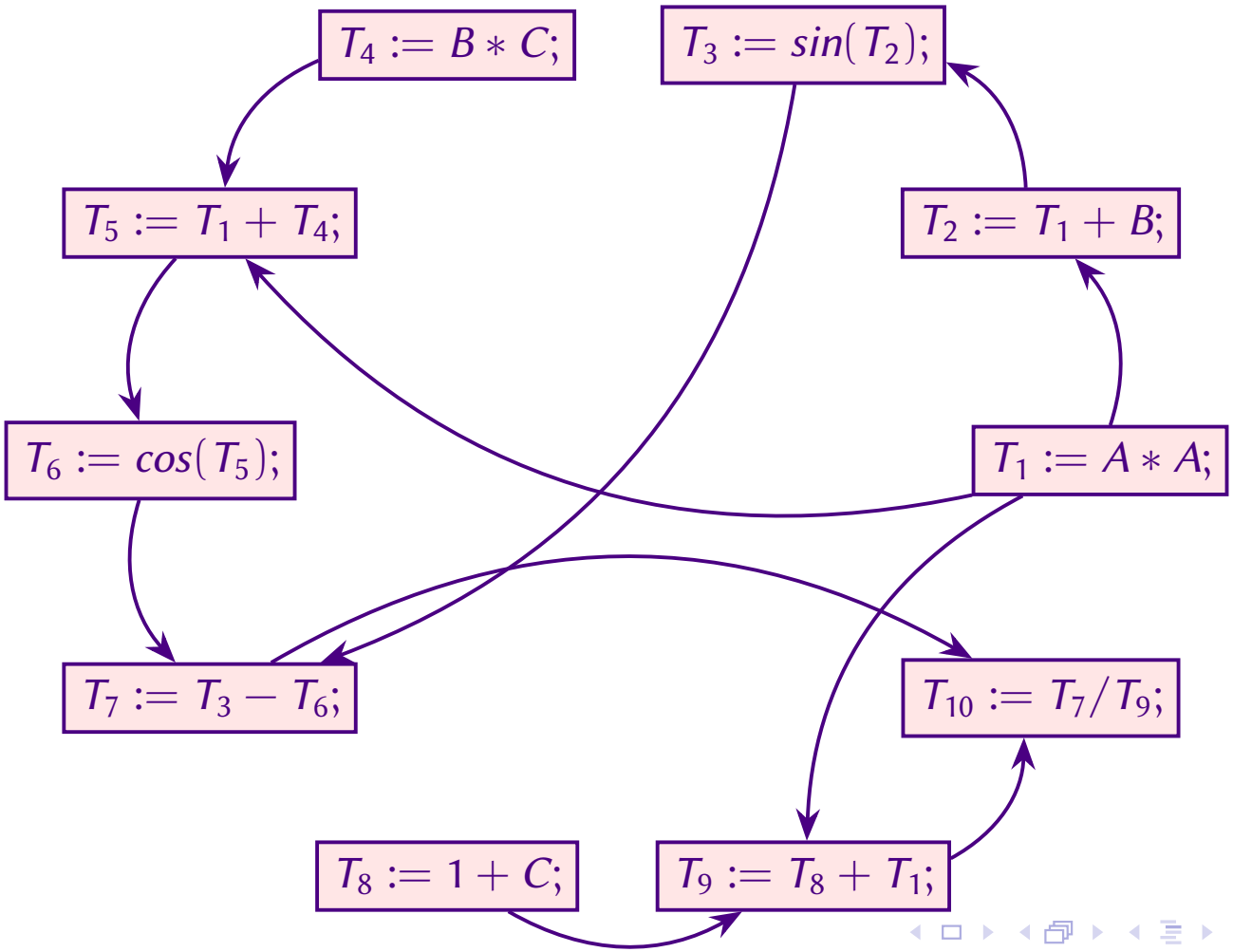
Solution

Décomposant tout d'abord le traitement en instructions comportant une expression à un ou deux opérandes, qui seront affectées à une variable temporaire. On trouve 10, qui formeraient, par exemple, le programme suivant sur un ordinateur monoprocesseur :

$$
\begin{array}{ll}
T_1 := A * A; & T_6 := \cos(T_5); \\
T_2 := T_1 + B; & T_7 := T_3 - T_6; \\
T_3 := \sin(T_2); & T_8 := 1 + C; \\
T_4 := B * C; & T_9 := T_8 + T_1; \\
T_5 := T_1 + T_4; & T_{10} := T_7 / T_9;
\end{array}
$$

Solution

Pour tirer parti du parallélisme, définissons un graphe orienté $G(X, E)$. Où X désigne l'ensemble des instructions. E contient l'arc (x, y) si le résultat de l'instruction x est nécessaire à l'instruction y (x doit donc précéder y). Soit le graphe $G(X, E)$ ainsi définie.



Solution

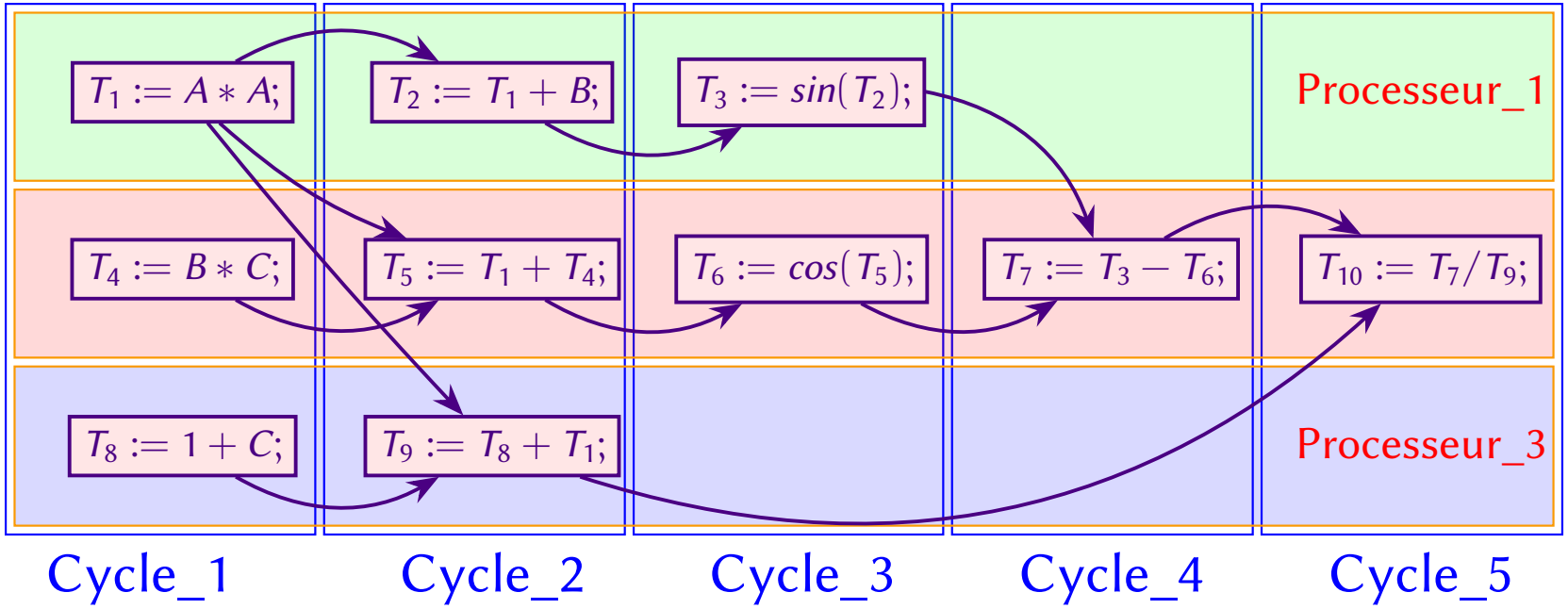
Réalisons un tri topologique de ce graphe.

X	$\Gamma^{-}(X)$
T_1	$\emptyset$
T_2	T_1
T_3	T_2
T_4	$\emptyset$
T_5	T_1, T_4

X	$\Gamma^{-}(X)$
T_6	T_5
T_7	T_3, T_6
T_8	$\emptyset$
T_9	T_1, T_8
T_{10}	T_7, T_9

On a donc la décomposition suivante : $N_1 = \{T_1, T_4, T_8\}$; $N_2 = \{T_2, T_5, T_9\}$; $N_3 = \{T_3, T_6\}$; $N_4 = \{T_7\}$; $N_5 = \{T_{10}\}$.

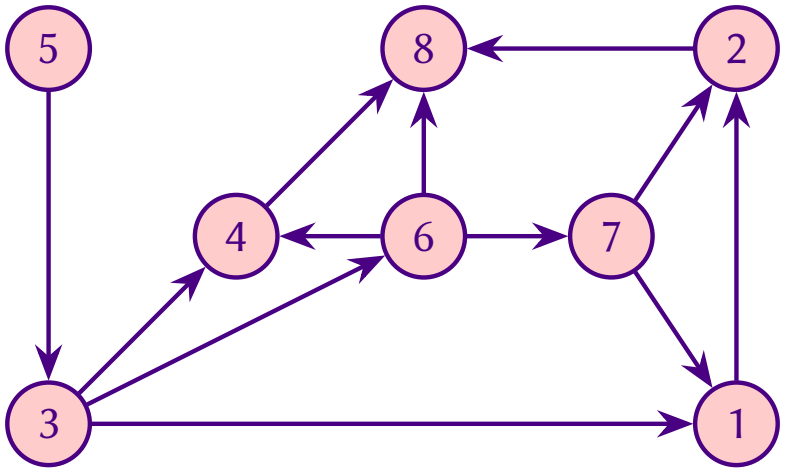
Solution



L'expression est donc calculable en 5 cycles machine et comme il y a au plus 3 instructions par cycle, 3 processeurs sont suffisants pour calculer l'expression.

Exercice (programmation)

Écrire un programme C++ qui réaliser le tri topologique du graphe suivant :



1. Le graphe est représenter par la matrice d'adjacence.
2. Le graphe est représenter par les listes d'adjacences.

Code : (la matrice d'adjacence)

```
1 #include <iostream>
2 #include <iomanip>
3 #include <list>
4
5 using namespace std;
6
7 #define n 8 //nombre de sommets du graphe
8
9 void initialisation_graphe(int adj[n][n])
10 {
11     adj[0][1] = 1;
12     adj[1][7] = 1;
13     adj[2][0] = 1;
14     adj[2][3] = 1;
15     adj[2][5] = 1;
16     adj[3][7] = 1;
17     adj[4][2] = 1;
18     adj[5][3] = 1;
19     adj[5][7] = 1;
20     adj[5][6] = 1;
21     adj[6][0] = 1;
22     adj[6][1] = 1;
23
24     return;
25 }
26

27 void Impression_Matrice_adj(int adj[n][n])
28 {
29     cout << endl
```

Code : (la matrice d'adjacence)

```
30          << "  La matrice d'adjacence du Graphe : ";
31     cout << endl
32          << endl;
33
34     cout << "\t     ";
35     for (int j = 0; j < n; j++)
36         cout << setw(2) << j + 1 << " ";
37     cout << endl;
38
39     cout << "\t    .";
40     for (int j = 0; j < n; j++)
41         cout << "...";
42     cout << endl;
43
44     for (int i = 0; i < n; i++) {
45         cout << " \t" << setw(2) << i + 1 << " : ";
46         for (int j = 0; j < n; j++)
47             cout << setw(2) << adj[i][j] << " ";
48         cout << endl;
49     }
50     return;
51 }
52


53 int main()
54 {
55     int graph[n][n] = { 0 }; /* Matrice d'adjacence du Graphe */
56     int degree[n] = { 0 }; /* contenant les degrés entrants des sommets */
57     int niveau[n] = { 0 }; /* niveau des sommets après le Tri Topologique */
```

Code : (la matrice d'adjacence)

```
58
59      int u;
60
61      initialisation_graphe(graph); /* initialisation de la Matrice d'adjacence */
62      Impression_Matrice_adj(graph); /* affichage de la Matrice d'adjacence */
63
64      /*
65         Parcours la matrice d'adjacence pour déterminer degré interieur
66         de chaque sommet du grahe                                        */
67
68      for (int j = 0; j < n; j++)
69          for (int i = 0; i < n; i++)
70              degree[i] = degree[i] + graph[j][i];
71
72      cout << endl
73           << "  Degre interieur des sommets : \n\n";
74
75      cout << "\t\t Sommet          : |";
76      for (int i = 0; i < n; i++)
77          cout << setw(2) << i + 1 << " |";
78      cout << endl;
79
80      cout << "\t\t degré interieur : |";
81      for (int i = 0; i < n; i++)
82          cout << setw(2) << degree[i] << " |";
83      cout << endl
84           << endl;
85
86      /* ***************************** debut du Tri Topologique *************** */
87
```

Code : (la matrice d'adjacence)

```
88      cout << endl
89           << "  Tri Topologique du Graphe :"
90           << " ";
91
92      list<int> F; /* Soit F une file d'attente initialement vide */
93      for (int i = 0; i < n; i++)
94          if (degree[i] == 0) {
95              F.push_back(i);
96              niveau[i] = 0;
97          }
98
99      while (!F.empty()) {
100         u = F.front();
101         F.pop_front();
102         cout << "  " << u + 1;
103
104         for (int v = 0; v < n; v++)
105             if (graph[u][v] == 1) {
106                 degree[v]--;
107                 if (degree[v] == 0) {
108                     F.push_back(v);
109                     if (niveau[v] < niveau[u] + 1)
110                         niveau[v] = niveau[u] + 1;
111                 }
112             }
113     }
114
115     /* ********************** fin du Tri Topologique ********************** */
116
```

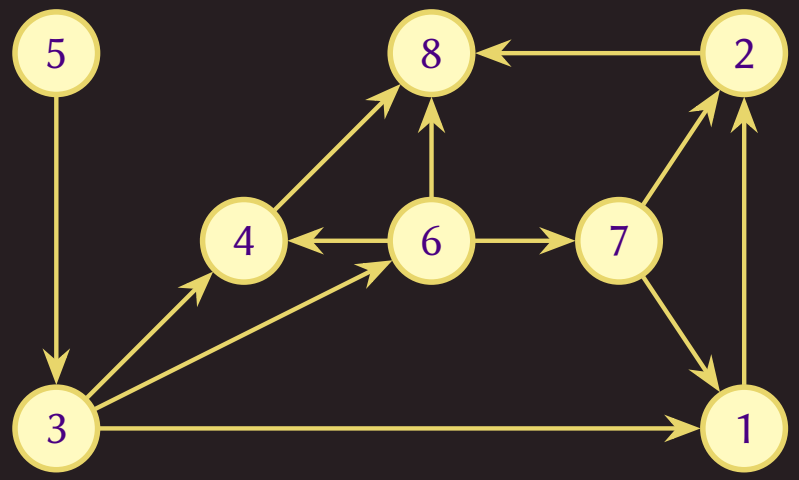
Code : (la matrice d'adjacence)

```
117     cout << endl
118          << endl
119          << "  Niveau des sommets :" << endl
120          << endl;
121
122     cout << "\t\t Sommet : |";
123     for (int i = 0; i < n; i++)
124         cout << setw(2) << i + 1 << " |";
125     cout << endl;
126
127     cout << "\t\t niveau : |";
128     for (int i = 0; i < n; i++)
129         cout << setw(2) << niveau[i] << " |";
130     cout << endl
131          << endl
132          << endl;
133
134     return 0;
135 }
```

Exécution

La matrice d'adjacence du Graphe :

	1	2	3	4	5	6	7	8
1 :	0	1	0	0	0	0	0	0
2 :	0	0	0	0	0	0	0	1
3 :	1	0	0	1	0	1	0	0
4 :	0	0	0	0	0	0	0	1
5 :	0	0	1	0	0	0	0	0
6 :	0	0	0	1	0	0	1	1
7 :	1	1	0	0	0	0	0	0
8 :	0	0	0	0	0	0	0	0



Degré intérieur des sommets :

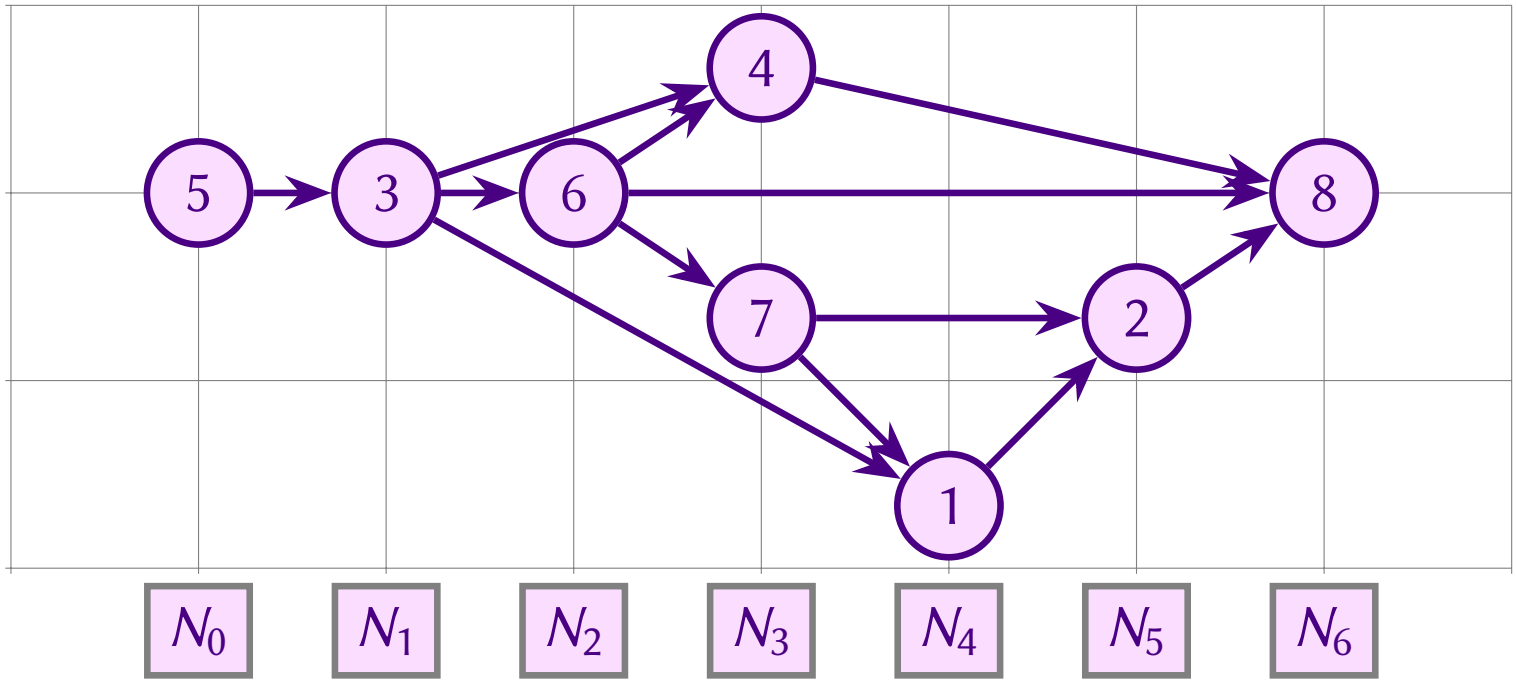
Sommet	1	2	3	4	5	6	7	8
degré intérieur	2	2	1	2	0	1	1	3

Tri Topologique du Graphe : 5 3 6 4 7 1 2 8

Niveau des sommets :

Sommet	1	2	3	4	5	6	7	8
niveau	4	5	1	3	0	2	3	6

Exécution



Code : (liste d'adjacence)

```
1 #include <iostream>
2 #include <vector>
3 #include <list>
4 #include <iomanip>
5
6 #define n 8 //nombre de sommets du graphe
7
8 using namespace std;
9
10 void initialisation_graphe(vector<list<int> >& adj)
11 {
12     adj[0].push_back(1);
13
14     adj[1].push_back(7);
15
16     adj[2].push_back(0);
17     adj[2].push_back(3);
18     adj[2].push_back(5);
19
20     adj[3].push_back(7);
21
22     adj[4].push_back(2);
23
24     adj[5].push_back(3);
25     adj[5].push_back(7);
26     adj[5].push_back(6);
```

Code : (liste d'adjacence)

```
27
28      adj[6].push_back(0);
29      adj[6].push_back(1);
30
31      return;
32 }
33

34 void Impression_Liste_adj(vector<list<int> > graphe)
35 {
36      cout << endl
37           << "  Le Graphe représenter par les listes d'adjacences : ";
38      cout << endl
39           << endl;
40      list<int>::iterator iter;
41      for (unsigned i = 0; i < graphe.size(); i++) {
42          cout << "                "
43               << "(" << i + 1 << ") : ";
44          for (iter = graphe[i].begin(); iter != graphe[i].end(); ++iter)
45              cout << "(" << *iter + 1 << "), ";
46          cout << "/" << endl;
47      }
48 }
49
```

Code : (liste d'adjacence)

```
50 void Calcul_Degre(vector<list<int> > graphe, int deg[n])
51 {
52     list<int>::iterator iter;
53     for (unsigned i = 0; i < graphe.size(); i++) {
54         for (iter = graphe[i].begin(); iter != graphe[i].end(); ++iter)
55             deg[*iter]++;
56     }
57     cout << endl
58          << "  Degré interieur des sommets : \n\n";
59
60     cout << "\t\t Sommet          : |";
61     for (int i = 0; i < n; i++)
62         cout << setw(2) << i + 1 << " |";
63     cout << endl;
64
65     cout << "\t\t degré interieur : |";
66     for (int i = 0; i < n; i++)
67         cout << setw(2) << deg[i] << " |";
68     cout << endl
69          << endl;
70 }
71
```

Code : (liste d'adjacence)

```
72 int main()
73 {
74     vector<list<int> > graphe(n);
75     int degree[n] = { 0 }; /* contenant les degrés entrants des sommets */
76     int niveau[n] = { 0 }; /* niveau des sommets après le Tri Topologique */
77
78     initialisation_graphe(graphe); /* initialisation des listes d'adjacences */
79
80     Impression_Liste_adj(graphe); /* Impression des listes d'adjacences */
81
82     Calcul_Degre(graphe, degree); /* Calcul des degrés des sommets du graphe */
83
84     /* ***************************** debut du Tri Topologique *************** */
85
86     cout << endl
87          << "  Tri Topologique du Graphe :"
88          << " ";
89     int v;
90     int u;
91     list<int>::iterator iter;
92
93     list<int> F; /* Soit F une file initialement vide */
94
95     for (unsigned i = 0; i < graphe.size(); i++)
96         if (degree[i] == 0) {
97             F.push_back(i);
```

Code : (liste d'adjacence)

```
98              niveau[i] = 0;
99          }
100
101     while (!F.empty()) {
102         u = F.front();
103         F.pop_front();
104         cout << " (" << u + 1 << ")";

105         for (iter = graphe[u].begin(); iter != graphe[u].end(); ++iter) {
106             v = *iter;
107             degree[v]--;
108             if (degree[v] == 0) {
109                 F.push_back(v);
110                 if (niveau[v] < niveau[u] + 1)
111                     niveau[v] = niveau[u] + 1;
112             }
113         }
114     }
115
116     /* ********************** fin du Tri Topologique ********************** */
117
118     cout << endl
119          << endl
120          << "  Niveau des sommets :" << endl
121          << endl;
122
```

Code : (liste d'adjacence)

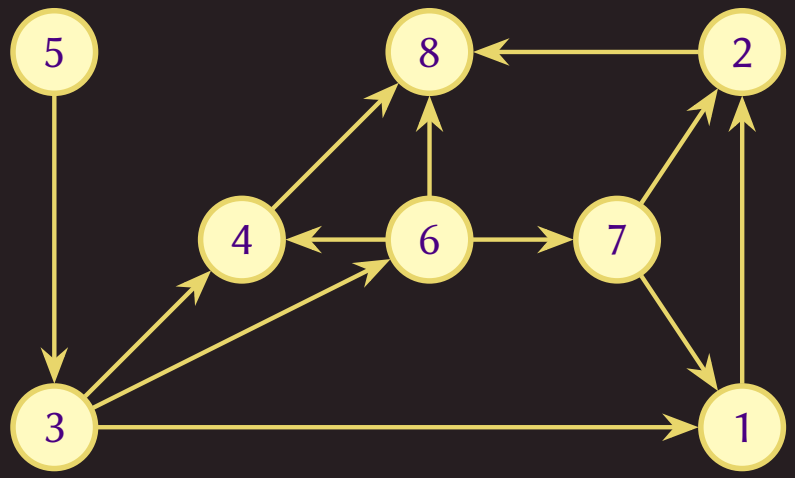
```
    cout << "\t\t Sommet : |";
    for (int i = 0; i < n; i++)
        cout << setw(2) << i + 1 << " |";
    cout << endl;

    cout << "\t\t niveau : |";
    for (int i = 0; i < n; i++)
        cout << setw(2) << niveau[i] << " |";
    cout << endl
         << endl
         << endl;
    return 0;
}
```

Exécution

Le Graphe représenter par les listes d'adjacences :

```
(1) : (2), /
(2) : (8), /
(3) : (1), (4), (6), /
(4) : (8), /
(5) : (3), /
(6) : (4), (8), (7), /
(7) : (1), (2), /
(8) : /
```



Degré intérieur des sommets :

Sommet	1	2	3	4	5	6	7	8
degré intérieur	2	2	1	2	0	1	1	3

Tri Topologique du Graphe : (5) (3) (6) (4) (7) (1) (2) (8)

Niveau des sommets :

Sommet	1	2	3	4	5	6	7	8
niveau	4	5	1	3	0	2	3	6

Merci de votre attention